

AI-Powered HR Analytics & Attrition Prediction System

Project submitted in partial fulfilment for the award of the degree of

Master of Computer Application

Submitted by

KULDEEP PATRA (2470185)

Under the Guidance of

PROF. PARTHASARATHI PATTNAYAK



School of Computer Application

KIIT University

April 2026

PROJECT TEAM

Name	Roll Number
KULDEEP PATRA	2470185
MD JUNAID FEROZ	2470195

Project Duration: 2nd January 2026 to 25th April 2026

School of Computer Application
KIIT University, Bhubaneswar

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled AI-Powered HR Analytics & Attrition Prediction System submitted to School of Computer Application, KIIT University in partial fulfilment of the requirement for the award of the degree of MASTER OF COMPUTER APPLICATIONS (MCA), is an authentic and original work carried out by **KULDEEP PATRA (2470185)** under my guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirements of any course of study.

Signature of the Student:

Signature of the Guide

Date:

Name:

Designation:

Date:

School of Computer Application
KIIT University, Bhubaneswar

CERTIFICATE

This is to certify that the project work entitled AI-Powered HR Analytics & Attrition Prediction System submitted by **KULDEEP PATRA (2470185)** is an authentic and original work.

Signature:

(Internal Examiner)

Date:

Signature:

(External Examiner)

Date:

DECLARATION

I, **KULDEEP PATRA, Roll No - (2470185)**, do hereby declare that the project report entitled **AI-Powered HR Analytics & Attrition Prediction System** submitted to **School of Computer Application, KIIT University**, Bhubaneswar for the award of the degree of **MASTER OF COMPUTER APPLICATION (MCA)**, is an authentic and original work carried out by me from 02/01/2026 to 25/04/ 2026 at School of Computer Application, KIIT University under the guidance of **Prof. Parthasarathi Pattnayak** .

Signature of the Student:

Date:

ACKNOWLEDGEMENT

The satisfaction which accompanies the successful completion of any task is incomplete without the mention of those persons whose guidance and support made it possible. Success is the epitome of hard work, perseverance, zeal, determination, and most importantly, encouraging guidance that serves as a beacon of light crowning our efforts with success.

I am grateful to the **Director General** and **Dean of School of Computer Application, KIIT Deemed to be University**, for their endless support and kind cooperation in the completion of this project. I am also thankful to **Prof. Parthasarathi Pattnayak** for the invaluable mentorship, insightful feedback, and constant encouragement throughout the duration of this project.

I extend my sincere gratitude to all the faculty members of the School of Computer Application, KIIT Deemed to be University, who have constantly strived to support us academically and professionally. Their expertise in machine learning, data analytics, and HR systems proved immensely valuable.

I would also like to thank my family and friends for their unwavering moral support and patience during the course of this project. Their encouragement during challenging times kept me motivated to deliver quality work.

Date:

Place:

.....
Signature of the Student

Name: KULDEEP PATRA
Roll No.: 2470185

ABSTRACT

In the modern business environment, employee attrition poses one of the most significant challenges to organisational productivity and financial stability. The cost of replacing a single employee, when accounting for recruitment, onboarding, and lost productivity, can range from fifty percent to two hundred percent of that employee's annual salary. Most organisations currently rely on reactive strategies such as exit interviews and annual surveys, which provide insights only after the decision to leave has already been made. This project presents the design and development of an AI-Powered HR Analytics and Attrition Prediction System that transforms this reactive approach into a proactive, data-driven strategy. The system is built on a four-layer architecture. The data layer utilises the IBM HR Employee Attrition Dataset containing 1,470 employee records across 35 features, stored in a Neon serverless PostgreSQL cloud database. The machine learning layer employs an XGBoost classifier trained to predict the binary outcome of employee attrition, achieving an overall accuracy of 86 percent and a ROC-AUC score of 0.7837. Model predictions are made interpretable through SHAP feature importance analysis, which identifies Overtime, Monthly Income, and Stock Option Level as the three strongest drivers of attrition. The application layer exposes the trained model and database capabilities through a Flask REST API, tunnelled to the public internet via ngrok from a Google Colab environment. The intelligence layer features an AI Agent powered by the LLaMA 3.3 70B large language model hosted on the Groq platform, which accepts natural language questions from HR professionals, converts them automatically to PostgreSQL queries, executes them against the live database, and returns plain-English analytical summaries. An interactive dashboard built with Power BI and Plotly provides visual analytics covering department-wise attrition rates, age versus risk scatter plots, income distributions, and overtime impact analysis. The entire system is deployed at zero cost using free-tier services, demonstrating that enterprise-grade AI-powered analytics are achievable without commercial licensing investment.

Keywords: Employee Attrition Prediction, XGBoost, SHAP, Natural Language to SQL, LLaMA 3.3 70B, Groq, Neon PostgreSQL, Flask, Power BI, HR Analytics.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
HR	Human Resources
API	Application Programming Interface
REST	Representational State Transfer
XGBoost	Extreme Gradient Boosting
SHAP	Shapley Additive exPlanations
ROC AUC	Receiver Operating Characteristic – Area Under Curve
SQL	Structured Query Language
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
GUI	Graphical User Interface
LLM	Large Language Model
NLP	Natural Language Processing
PCA	Principal Component Analysis
SMOTE	Synthetic Minority Over-sampling Technique
DFD	Data Flow Diagram
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol

INDEX

Chapter	Title	Page No.
Chapter 1	Introduction	1
1.1	Company Profile	11
1.2	Existing System and Need for System	11
1.3	Problems with the Existing System	12
1.4	Need for the Proposed System	6
Chapter 2	Proposed System	16
2.1	Proposed System	16
2.2	Objectives of System	17
2.3	User Requirements	18
Chapter 3	Analysis and Design	19
3.1	Object Diagram	19
3.2	Class Diagram	19
3.3	Use Case Diagrams	21
3.4	Module Hierarchy Diagram	23
3.5	Component Diagram	24
3.6	Deployment Diagram	25
3.7	Module Specifications	26
3.8	Interface Diagram	29
3.9	Web Site Map Diagram	30
3.10	User Interface Design	31
Chapter 4	User Manual	33
4.1	User Manual	33
4.2	Operations Manual / Menu Explanation	34
4.3	Program Specifications / Flow Charts	36

Chapter	Title	Page No.
—	Drawbacks and Limitations	40
—	Proposed Enhancements	41
—	Conclusion	42
—	Bibliography	43
Annexure 1	User Interface Screens	44
Annexure 2	Output Reports with Data	47
Annexure 3	Code	48
—	End of Report	60

INDEX

1.1 Company Profile

This project has been developed as an academic initiative under the School of Computer Applications (SCA), KIIT Deemed to be University, Bhubaneswar, Odisha. KIIT Deemed to be University is one of India's premier technical institutions, established in 1992 by Prof. Achyuta Samanta. KIIT received the status of a Deemed to be University in 2004 under Section 3 of the UGC Act and has since grown into a globally recognised research and teaching university with over 30,000 students from more than 60 countries.

The School of Computer Applications at KIIT is dedicated to producing technology professionals who are equipped with both theoretical knowledge and practical skills. The fourth semester of the MCA programme is entirely devoted to project-oriented work, under Course Code MC5184, requiring students to design and implement a complete software system under the supervision of a faculty guide. The present project was undertaken from 2nd January 2026 to 25th April 2026 under the guidance of Prof. [Supervisor Name], at the School of Computer Applications, KIIT University.

The project is an academic technical project and does not belong to a commercial organisation. The system developed is intended to demonstrate the application of machine learning, natural language processing, and cloud database technologies to the domain of human resources analytics. The two-member team comprised Student A, responsible for the machine learning pipeline and backend API, and Student B, responsible for the AI agent layer and analytics dashboard.

1.2 Existing System and Need for System

The Existing System

In most organisations today, the management of employee attrition is handled through a combination of manual processes and periodic surveys. The most commonly used tools in the existing system include annual employee engagement surveys, performance review cycles, exit interviews, and basic HR reporting dashboards that show historical attrition numbers.

The annual engagement survey is conducted once or twice a year and captures employee satisfaction across dimensions such as work environment, management quality, compensation satisfaction, and career growth opportunities. However, the results are aggregated at the team or department level and are rarely used for individual-level risk assessment. By the time the survey data is analysed and acted upon, the conditions that caused dissatisfaction may have worsened.

The exit interview is conducted after an employee has already resigned. While it provides qualitative insight into the reasons for departure, it arrives too late to influence the departure

decision. Research consistently shows that employees rarely disclose the true reason for leaving in formal exit interviews, further reducing the reliability of this data.

Manual HR tracking involves HR business partners maintaining spreadsheets of employee data, flagging employees who appear at risk based on subjective judgment. This approach is inconsistent, unscalable, and highly dependent on the experience and availability of individual HR professionals.

Basic reporting dashboards show historical attrition rates broken down by department, job level, or tenure band. These dashboards are descriptive — they show what has already happened — rather than predictive of what is likely to happen next.

Problems with the Existing System

The existing system suffers from several critical limitations. First, it is entirely reactive. Interventions are triggered only after an employee has resigned, rather than before the decision is made. Second, it lacks predictive capability. No component of the existing system assigns a quantitative risk score to individual employees based on their current attributes. Third, it is not scalable. Manual tracking by HR business partners cannot be realistically extended to organisations with thousands of employees. Fourth, the insights generated are not actionable at the individual level. Aggregate attrition rates tell management that a department is at risk but do not identify which specific employees should be prioritised for retention conversations. Fifth, querying HR data requires SQL knowledge or dependence on the IT department, making self-service analytics difficult for HR professionals.

Need for the Proposed System

Given these limitations, there is a clear and pressing need for a system that can predict employee attrition at the individual level before it occurs, explain why the model makes each prediction in terms that HR professionals can understand, allow HR managers to ask questions about the workforce in natural English without needing technical knowledge, and present key HR metrics in a visually accessible dashboard that can be updated on demand.

The AI-Powered HR Analytics and Attrition Prediction System addresses each of these needs through the integration of machine learning, explainable AI, a natural language query interface, and an interactive analytics dashboard.

1. INTRODUCTION

1.1 Background

In today's hyper-competitive business environment, human capital is widely recognized as the most critical asset of any organization. Employee attrition — the voluntary or involuntary departure of employees — poses one of the most persistent and costly challenges to companies across industries. Research by the Society for Human Resource Management (SHRM) estimates that replacing a single employee can cost between 50% and 200% of that employee's annual salary when accounting for recruitment, onboarding, training, and lost productivity.

Traditional HR departments have historically relied on exit interviews, manual surveys, and retrospective data analysis to understand why employees leave. These approaches are reactive by nature, providing insights only after valuable employees have already decided to depart. The rise of big data, machine learning, and artificial intelligence has created a paradigm shift, enabling organizations to predict attrition proactively and intervene before the departure occurs.

AI-powered HR analytics transforms raw employee data — encompassing demographics, job roles, compensation, satisfaction scores, performance metrics, and engagement levels — into actionable intelligence. By training predictive models on historical attrition patterns, organizations can identify at-risk employees with high accuracy, enabling targeted retention strategies that improve employee satisfaction and reduce turnover costs.

1.2 Problem Statement

Despite the availability of sophisticated HR information systems and large volumes of employee data, most organizations still struggle to predict and prevent employee attrition effectively. Key problems include:

1. **Reactive Approach:** Current HR strategies are predominantly reactive, addressing attrition after it occurs rather than predicting and preventing it in advance.
2. **Data Silos:** Employee data is fragmented across multiple systems — HRMS, payroll, performance management, and engagement platforms — making holistic analysis difficult.
3. **Lack of Explainability:** Even when predictive models exist, HR professionals often cannot understand why a particular employee is flagged as at-risk, making it difficult to design effective interventions.
4. **Inaccessibility to Non-Technical Users:** Machine learning models are typically accessible only to data scientists, excluding HR managers and business leaders who make retention decisions.
5. **Scalability:** Manual analysis of employee attrition patterns becomes increasingly impractical as organizations grow, necessitating automated, scalable solutions.

This project addresses these challenges by developing an integrated AI-powered platform that predicts employee attrition, explains the predictions using SHAP values, and provides an intuitive natural language interface powered by a large language model.

1.3 Objectives

The following objectives guide the development of the AI-Powered HR Analytics & Attrition Prediction System:

- To develop a robust machine learning pipeline for employee attrition prediction using the XGBoost classifier on the IBM HR Employee Attrition Dataset.
- To implement comprehensive data preprocessing including missing value handling, label encoding, and feature engineering to improve model performance.
- To store processed HR data in a scalable cloud-based Neon PostgreSQL database for persistent data management and retrieval.
- To evaluate model performance using industry-standard metrics including ROC AUC, precision, recall, F1-score, and confusion matrix analysis.
- To integrate SHAP-based model explainability, enabling HR professionals to understand the factors driving individual attrition predictions.
- To deploy the predictive model as a RESTful Flask API supporting real-time attrition risk assessment.
- To integrate a Groq Llama AI conversational agent enabling natural language queries for attrition insights by non-technical HR staff.
- To expose the system externally via Ngrok tunnelling for accessible deployment and demonstration.
- To provide actionable HR insights and recommendations based on feature importance and SHAP analysis.

1.4 Project Features

Feature	Description
Attrition Prediction	Binary classification model predicting employee likelihood of leaving
Data Preprocessing Pipeline	Automated handling of missing values, encoding, and feature scaling
Cloud Database Integration	Neon PostgreSQL for scalable, persistent data storage
SHAP Explainability	Per-employee explanation of attrition risk factors
Feature Importance Dashboard	Visual ranking of top attrition predictors
RESTful Flask API	HTTP endpoints for real-time prediction requests

Feature	Description
Groq Llama AI Agent	Natural language interface for HR queries
Ngrok Deployment	Secure external tunnel for system accessibility
ROC AUC Evaluation	Industry-standard model performance metrics
CSV Data Export	Export of predictions and analytics for reporting

1.5 Scope and Limitations

1.5.1 Scope of Project

The scope of this project encompasses the complete lifecycle of an AI-powered HR analytics system, from raw data ingestion to deployment of a user-accessible predictive service. Specifically, the system covers:

- Data ingestion and preprocessing of the IBM HR Employee Attrition Dataset comprising 1,470 employee records and 35 features.
- Design and implementation of an XGBoost binary classification model with hyperparameter optimization.
- Integration with a Neon PostgreSQL cloud database for data persistence and scalability.
- Deployment of a Flask-based REST API with Ngrok external exposure.
- Integration of the Groq Llama 3 conversational AI for natural language HR queries.
- SHAP-based model explainability for both global feature importance and local per-employee predictions.
- Generation of HR analytics reports and recommendations based on model insights.

1.5.2 Limitations of Project

- The system is trained on the IBM HR Attrition Dataset, which, while widely used in academic research, may not perfectly represent all industry-specific attrition patterns.
- The model does not currently support multi-class attrition categorization (e.g., voluntary vs. involuntary departure).
- Real-time integration with live HRMS systems requires additional API connectors not included in the current scope.
- The Groq Llama AI agent is dependent on external API availability and may have latency in high-traffic scenarios.
- The system does not currently support incremental learning as new employee data becomes available, requiring periodic model retraining.

CHAPTER 2: PROPOSED SYSTEM

2.1 Introduction

The proposed system is an AI-Powered HR Analytics and Attrition Prediction System that replaces the reactive, manual approach to employee attrition management with a proactive, data-driven platform. The system integrates machine learning for prediction, explainable AI for transparency, a cloud database for data management, a REST API for system access, a large language model for natural language querying, and a business intelligence dashboard for visual analytics — all within a single, cohesive architecture operating at zero cost.

System Architecture Overview

The system is organised into four distinct layers, each with well-defined responsibilities and interfaces.

The Data Layer is responsible for all persistent data storage and retrieval. The IBM HR Employee Attrition Dataset, consisting of 1,470 employee records with 35 features, is cleaned, encoded, and loaded into a Neon serverless PostgreSQL database table named employees. Following machine learning model training, a derived column named attrition_risk (a floating-point probability between 0 and 1) is added to this table through a bulk update operation. The data layer serves as the single source of truth for all downstream components.

The Machine Learning Layer is responsible for building the predictive model. An XGBoost classifier is trained on 80 percent of the dataset (1,176 records) and evaluated on the remaining 20 percent (294 records). The trained model is serialised as a model.pkl file for persistent storage. The SHAP Tree Explainer generates feature importance values that explain which factors most influence each prediction. The ML layer produces both a production-ready prediction artefact and interpretability outputs for HR professionals.

The Application Layer is a Flask REST API that acts as the interface between the data and ML layers on one side, and the presentation layer on the other. It exposes five HTTP endpoints. The POST /predict endpoint accepts a JSON object containing an employee's feature values and returns the predicted attrition probability along with a risk classification. The POST /ask endpoint accepts a natural language question and returns a SQL query, the resulting data rows, and a plain-English analytical answer. The GET /kpi-summary endpoint returns aggregated HR key performance indicators. The GET /high-risk endpoint returns the ten current employees with the highest predicted attrition risk. The GET /health endpoint confirms API availability.

The Intelligence and Presentation Layer combines the AI Agent and the user-facing dashboards. The AI Agent uses LLaMA 3.3 70B via Groq to perform a two-stage pipeline: in the first stage it converts the user's natural language question to a PostgreSQL query; in the second stage it converts the query results to a concise strategic HR insight. The browser-based chat interface renders this conversation in a dark-themed UI with a KPI summary bar and preset

demo question. The Power BI and Plotly dashboards provide visual analytics for management-level reporting.

The entire system runs inside Google Colab with the Flask server exposed via an ngrok tunnel. Google Drive provides persistence for the model.pkl artefact across notebook sessions.

2.2 Objectives of System

The following specific objectives were defined and achieved in this project:

Objective 1: To acquire, load, and perform exploratory data analysis on the IBM HR Employee Attrition Dataset, understanding the class distribution, feature types, and correlation structure.

Objective 2: To preprocess the dataset by detecting missing values, encoding all categorical string features using Label Encoder, and persisting the cleaned data to a Neon PostgreSQL cloud database.

Objective 3: To train an XGBoost binary classifier on the pre-processed employee data, targeting an overall accuracy above 85 percent and a ROC-AUC score above 0.75 on the held-out test set.

Objective 4: To evaluate the trained model comprehensively using a confusion matrix, classification report including precision, recall, and F1-score for both classes, and ROC-AUC score.

Objective 5: To generate SHAP feature importance visualisations using the TreeExplainer, providing interpretable, per-feature explanations of the model's predictions that HR professionals can act upon.

Objective 6: To develop and expose a Flask REST API from within Google Colab, using ngrok tunnelling to make all endpoints publicly accessible, and to build a POST /predict endpoint that serves real-time attrition risk predictions.

Objective 7: To develop an AI Agent that accepts natural language HR questions, uses LLaMA 3.3 70B via the Groq API to generate syntactically correct PostgreSQL queries incorporating Label Encoding column mappings, executes those queries against Neon, and returns concise plain-English analytical answers.

Objective 8: To build an interactive HR analytics dashboard in Power BI and Plotly that presents department attrition rates, age versus risk scatter plots, income distribution, and overtime impact, along with a browser-based AI Agent chat interface for non-technical HR users.

2.3 User Requirements

The system serves three distinct types of users, each with different technical backgrounds and different needs.

HR Manager (Primary End User)

The HR Manager is a non-technical professional responsible for workforce planning, employee engagement, and retention strategy. This user has no knowledge of SQL, Python, or machine learning. The HR Manager requires the ability to view key workforce KPIs such as total headcount, overall attrition rate, average tenure, and the number of employees currently classified as high risk, through a visual dashboard that requires no technical operation. The HR Manager requires the ability to ask questions about employee attrition data in plain English and receive answers within seconds. Sample questions this user would ask include "Which department has the highest attrition rate?", "Show me the top five employees most at risk of leaving," "How does working overtime affect attrition?", and "What is the average salary of employees who have left versus those who stayed?" The HR Manager requires the answer to be presented in both a data table and a natural language summary that can be directly shared in management meetings. The HR Manager does not require access to code, notebooks, or API documentation.

Data Scientist / ML Engineer (Technical User)

The Data Scientist is a technical user responsible for maintaining and improving the predictive model. This user requires access to the full Colab notebook environment, the ability to retrain the model with updated data, and access to the SHAP summary plot to monitor which features are driving predictions over time. The Data Scientist requires clear model evaluation metrics including the confusion matrix, classification report, and ROC-AUC score. This user also requires the ability to modify the XGBoost hyperparameters and LLM system prompts, and to update the Label Encoding mappings in the AI Agent when the dataset schema changes.

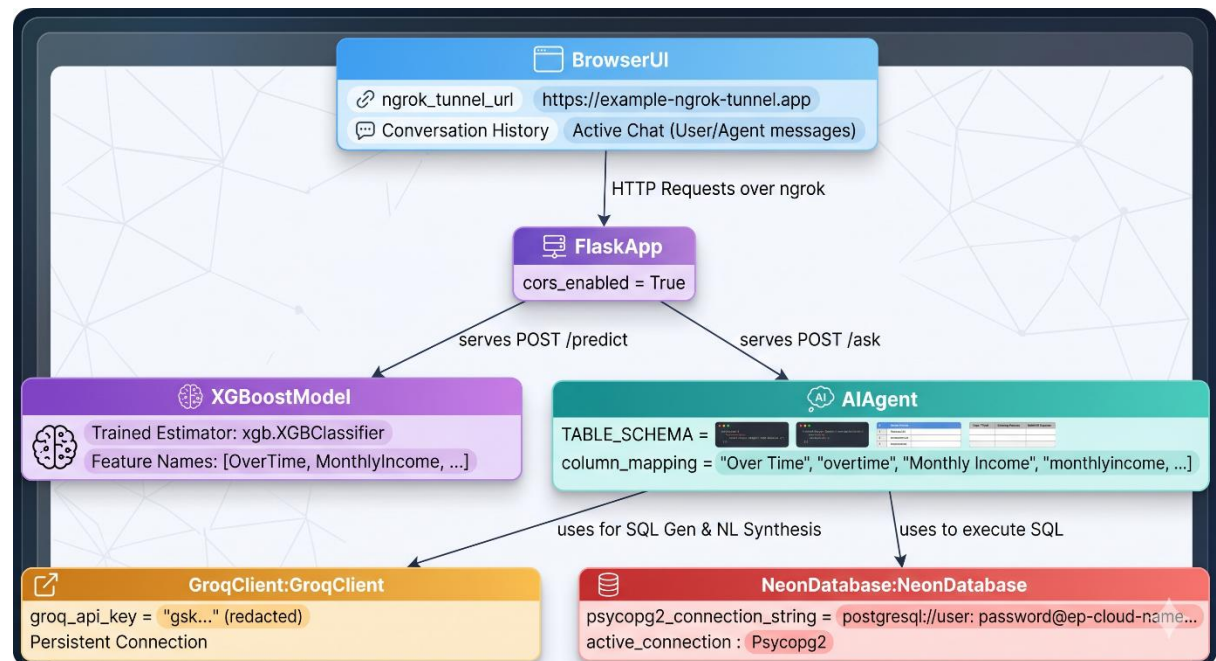
System Administrator (Operational User)

The System Administrator is responsible for starting and maintaining the running system. This user requires clear instructions for executing the Colab notebooks in sequence, setting the Groq API key and Neon connection string, starting the Flask background server, and obtaining the ngrok public URL to share with HR Manager users. The System Administrator also requires access to the GET /health endpoint to confirm that the API is running correctly before demonstrations.

CHAPTER 3: ANALYSIS AND DESIGN

3.1 Object Diagram

The Object Diagram for the AI-Powered HR Analytics and Attrition Prediction System depicts the runtime instances of the major classes and their relationships during a typical user interaction session.

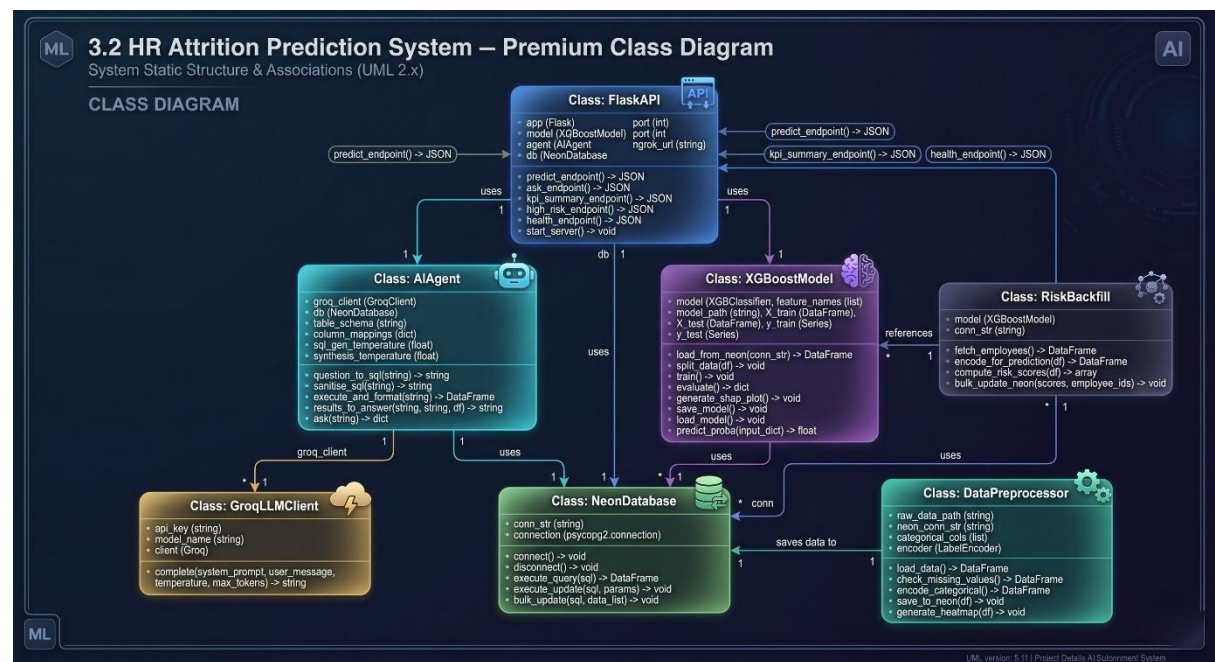


The key runtime objects in the system are as follows. A Groq Client object is instantiated with the Groq API key and maintains a persistent connection to the Groq inference service. An AI Agent object holds a reference to the GroqClient and to the Neon Database object, and contains the active TABLE_SCHEMA string and column mapping dictionaries as instance attributes. A Neon Database object holds the active psycopg2 connection string and provides query execution methods. A FlaskApp object is instantiated with CORS enabled and holds references to the loaded XGBoostModel and AI Agent objects. The XGBoostModel object is loaded from the model.pkl file and holds the trained estimator along with the feature_names_in_ attribute used during inference. A Browser UI object represents the client-side HTML/JavaScript application running in the user's browser, holding the current API_BASE URL and the conversation history as an array of message objects.

The following object relationships exist at runtime. The Flask App object uses the XGBoostModel object to serve POST /predict requests. The Flask App object uses the AI Agent object to serve POST /ask requests. The AI Agent object uses the GroqClient object for both SQL generation and natural language synthesis. The AI Agent object uses the Neon Database object to execute generated SQL queries. The Browser UI object communicates with the Flask object exclusively through HTTP requests to the ngrok tunnel URL.

3.2 Class Diagram

The Class Diagram describes the static structure of the system, showing the major classes, their attributes, their methods, and the relationships between them.



Class: DataPreprocessor

Attributes: raw_data_path (string), neon_conn_str (string), categorical_cols (list), encoder (LabelEncoder)

Methods: load_data() returns DataFrame, check_missing_values() returns DataFrame, encode_categorical() returns DataFrame, save_to_neon(df) returns void, generate_heatmap(df) returns void

Class: XGBoostModel

Attributes: model (XGBClassifier), feature_names (list), model_path (string), X_train (DataFrame), X_test (DataFrame), y_train (Series), y_test (Series)

Methods: load_from_neon(conn_str) returns DataFrame, split_data(df) returns void, train() returns void, evaluate() returns dict, generate_shap_plot() returns void, save_model() returns void, load_model() returns void, predict_proba(input_dict) returns float

Class: RiskBackfill

Attributes: model (XGBoostModel), conn_str (string)

Methods: `fetch_employees()` returns DataFrame, `encode_for_prediction(df)` returns DataFrame, `compute_risk_scores(df)` returns array, `bulk_update_neon(scores, employee_ids)` returns void

Class: NeonDatabase

Attributes: `conn_str` (string), `connection` (psycopg2.connection)

Methods: `connect()` returns void, `disconnect()` returns void, `execute_query(sql)` returns DataFrame, `execute_update(sql, params)` returns void, `bulk_update(sql, data_list)` returns void

Class: AIAgent

Attributes: `groq_client` (GroqClient), `db` (NeonDatabase), `table_schema` (string), `column_mappings` (dict), `sql_gen_temperature` (float), `synthesis_temperature` (float)

Methods: `question_to_sql(question)` returns string, `sanitise_sql(raw_sql)` returns string, `execute_and_format(sql)` returns DataFrame, `results_to_answer(question, sql, df)` returns string, `ask(question)` returns dict

Class: GroqLLMClient

Attributes: `api_key` (string), `model_name` (string), `client` (Groq)

Methods: `complete(system_prompt, user_message, temperature, max_tokens)` returns string

Class: FlaskAPI

Attributes: `app` (Flask), `model` (XGBoostModel), `agent` (AIAgent), `db` (NeonDatabase), `port` (int), `ngrok_url` (string)

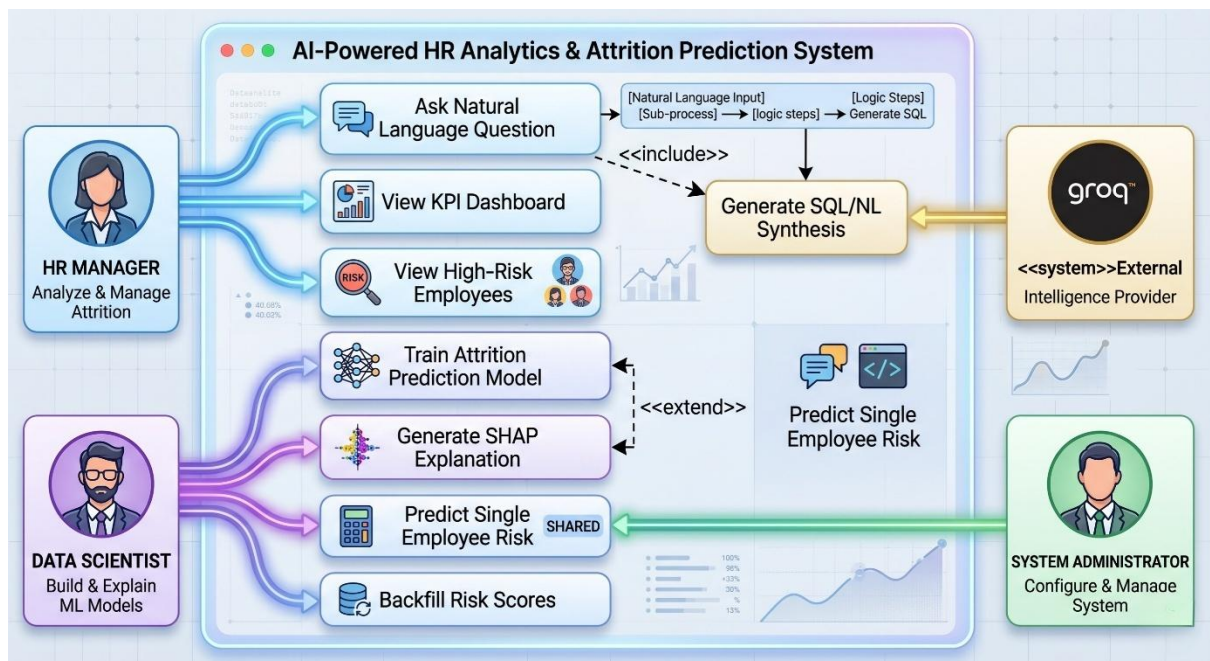
Methods: `predict_endpoint()` returns JSON, `ask_endpoint()` returns JSON, `kpi_summary_endpoint()` returns JSON, `high_risk_endpoint()` returns JSON, `health_endpoint()` returns JSON, `start_server()` returns void

3.3 Use Case Diagrams

Actors in the System

The system has four actors. The HR Manager is the primary human actor who interacts with the AI Agent chat interface and views dashboards. The Data Scientist is a technical actor who executes the Colab notebooks to train models and run the system. The System Administrator is an actor responsible for configuring and starting the API server. The Groq LLM Service is an external system actor that provides SQL generation and natural language synthesis capabilities.

Use Cases



Use Case 1: Ask Natural Language Question Actor: HR Manager Description: The HR Manager types a question in plain English into the chat interface. The system passes the question to the AI Agent, which generates a SQL query, executes it against Neon, and returns a table of results along with a plain-English summary. Precondition: Flask API is running and AI Agent is initialised with valid Groq API key. Postcondition: The chat interface displays the SQL used, the data table, and the AI-generated answer.

Use Case 2: View KPI Dashboard Actor: HR Manager Description: The HR Manager opens the chat interface, which automatically loads KPI cards showing total employees, attrition rate, high-risk count, and average salary. Precondition: Flask API /kpi-summary endpoint is accessible. Postcondition: KPI bar is populated with current values from Neon.

Use Case 3: Train Attrition Prediction Model Actor: Data Scientist Description: The Data Scientist executes Phase 2 of the Colab notebook, which fetches data from Neon, trains the XGBoost model, evaluates it, and saves it as model.pkl. Precondition: Phase 1 has been completed and employees table exists in Neon. Postcondition: model.pkl is saved to Google Drive and evaluation metrics are printed.

Use Case 4: Generate SHAP Explanation Actor: Data Scientist Description: The Data Scientist executes the SHAP cell in the Phase 2 notebook to generate the feature importance beeswarm plot. Precondition: XGBoost model is trained and X_test is available. Postcondition: SHAP summary plot is displayed in the notebook output.

Use Case 5: Predict Single Employee Risk Actor: System Administrator or Data Scientist Description: An HTTP POST request is sent to the /predict endpoint with a JSON body containing employee feature values. The API returns the predicted attrition probability and a

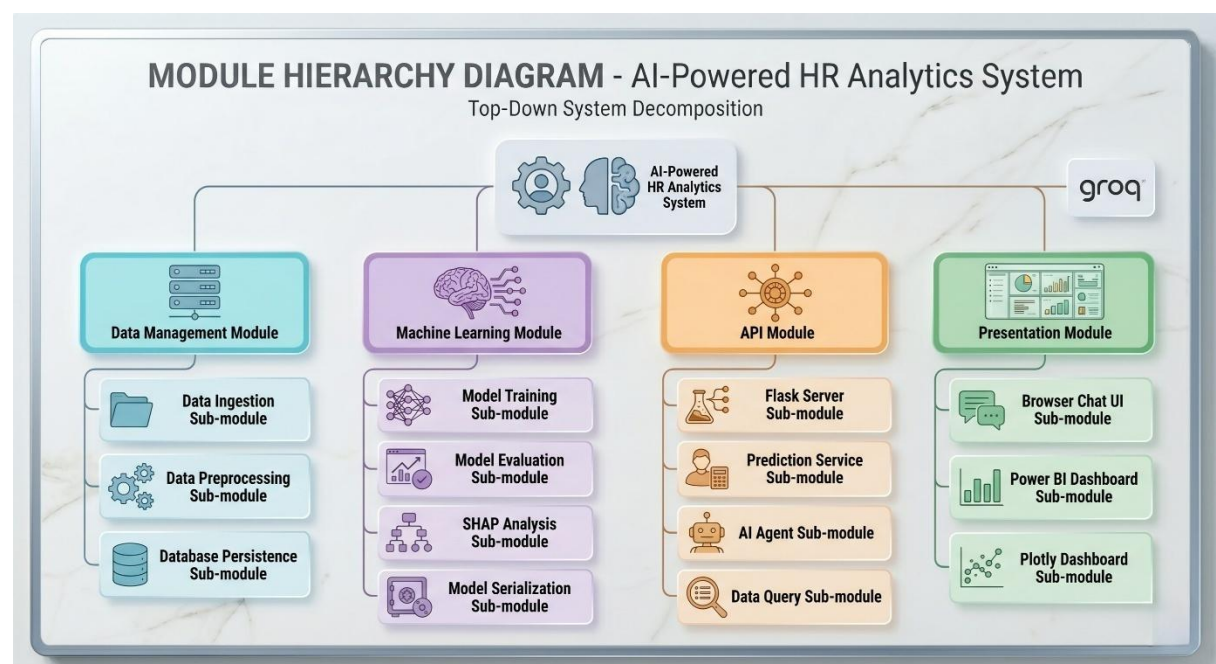
risk classification. Precondition: Flask API is running and model.pkl is loaded. Postcondition: JSON response with attrition_probability and will_attrite is returned.

Use Case 6: Backfill Risk Scores Actor: Data Scientist Description: The Data Scientist runs the backfill script which loads the trained model, retrieves all employees from Neon, computes risk scores for each, and performs a bulk SQL UPDATE. Precondition: model.pkl exists and employees table exists in Neon. Postcondition: attrition_risk column is populated for all 1,470 employees.

Use Case 7: View High-Risk Employees Actor: HR Manager Description: The HR Manager clicks the preset "Top risk employees" button in the chat interface, which calls the /high-risk endpoint and returns the top ten current employees by predicted attrition risk. Precondition: attrition_risk column has been backfilled. Postcondition: Table of ten highest-risk employees is displayed.

3.4 Module Hierarchy Diagram

The Module Hierarchy Diagram shows the top-down decomposition of the system into its constituent modules.



At the top level is the AI-Powered HR Analytics System. This decomposes into four second-level modules: Data Management Module, Machine Learning Module, Application Programming Interface Module, and Presentation Module.

The Data Management Module decomposes into: Data Ingestion Sub-module (CSV loading, missing value check), Data Preprocessing Sub-module (categorical encoding, constant

column removal), and Database Persistence Sub-module (SQLAlchemy engine, Neon write/read operations, bulk update operations).

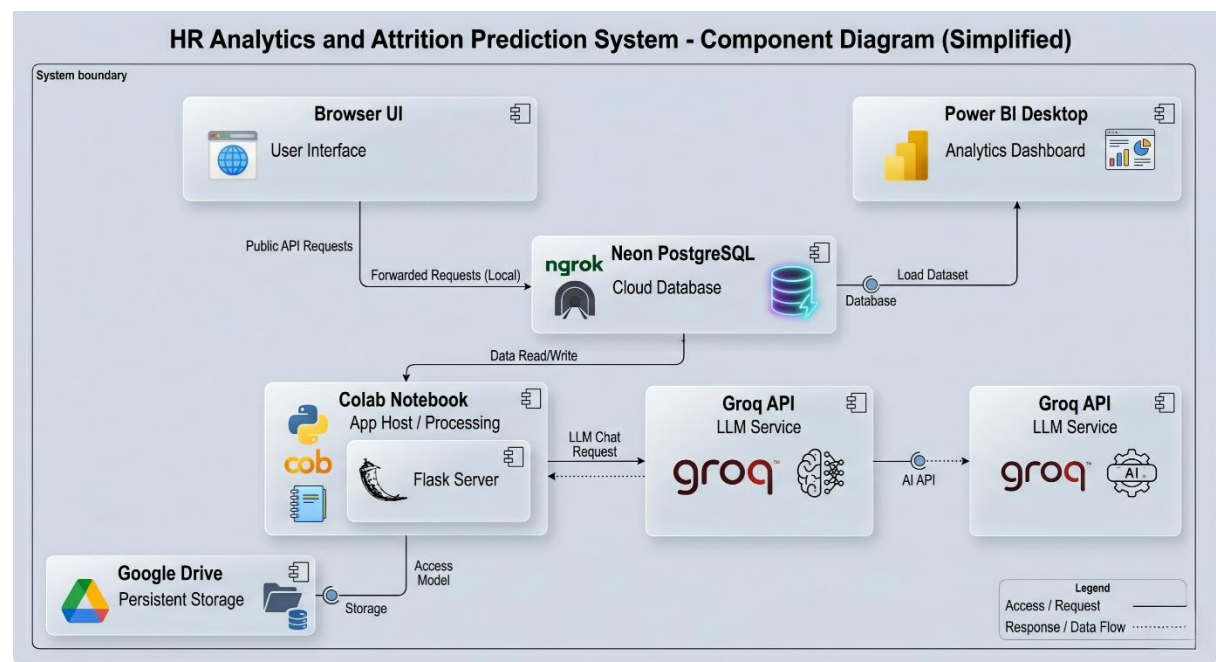
The Machine Learning Module decomposes into: Model Training Sub-module (XGBoost classifier, train/test split), Model Evaluation Sub-module (confusion matrix, classification report, ROC-AUC), SHAP Analysis Sub-module (TreeExplainer, summary plot generation), and Model Serialisation Sub-module (joblib dump/load).

The API Module decomposes into: Flask Server Sub-module (route definitions, CORS configuration, background threading), Prediction Service Sub-module (/predict endpoint, feature vector validation), AI Agent Sub-module (SQL generation pipeline, NL synthesis pipeline), and Data Query Sub-module (/kpi-summary, /high-risk, /health endpoints).

The Presentation Module decomposes into: Browser Chat UI Sub-module (HTML/CSS/JS, fetch API calls, table rendering), Power BI Dashboard Sub-module (CSV export, visual design), and Plotly Dashboard Sub-module (executive charts, KPI annotations).

3.5 Component Diagram

The Component Diagram shows the physical components of the system and their interfaces.



The system comprises the following components:

The Colab Notebook Component contains the Python execution environment and hosts the Flask server process. It exposes the HTTP interface to the ngrok tunnel. It depends on the Neon PostgreSQL component, the Groq API component, and the Google Drive storage component.

The ngrok Tunnel Component acts as a reverse proxy, receiving HTTPS requests from the browser and forwarding them as HTTP requests to the local Flask server on port 5040. It exposes a public HTTPS URL.

The Neon PostgreSQL Component is a cloud-hosted serverless PostgreSQL database. It exposes a standard PostgreSQL wire protocol interface on port 5432 with TLS encryption. The Colab notebook connects to it via psycopg2 and SQLAlchemy.

The Groq API Component is a cloud-hosted LLM inference service. It exposes an OpenAI-compatible chat completions REST API. The Colab notebook connects to it via the Groq Python SDK over HTTPS.

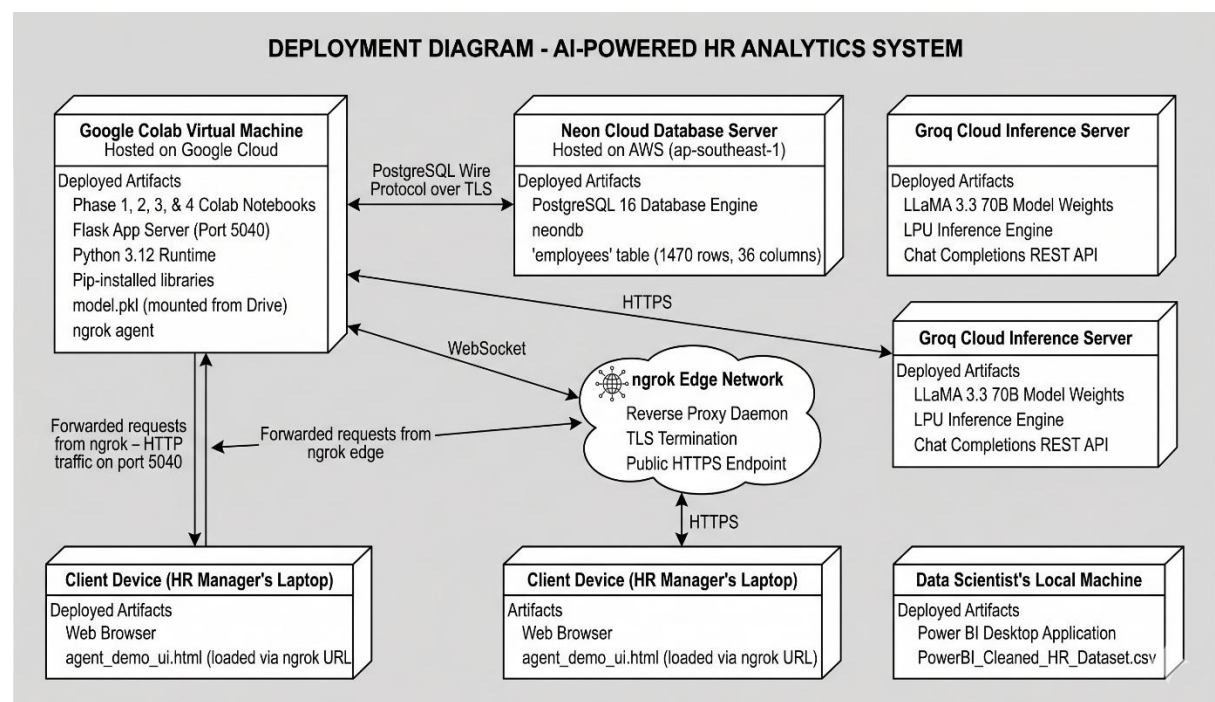
The Google Drive Component provides persistent file storage for model.pkl across Colab sessions. It is accessed via the google.colab.drive library.

The Browser UI Component is a static HTML/CSS/JavaScript application running in the user's browser. It communicates with the Flask API exclusively through HTTPS requests to the ngrok URL. It renders KPI cards, chat messages, and data tables.

The Power BI Desktop Component is a local desktop application that loads the PowerBI_Cleaned_HR_Dataset.csv exported from Neon and renders the analytics dashboard.

3.6 Deployment Diagram

The Deployment Diagram shows the physical nodes on which the system components are deployed and the communication paths between them.



The deployment involves the following nodes:

Node 1: Google Colab Virtual Machine (hosted on Google's infrastructure) Deployed artefacts: Phase 1, 2, 3, and 4 Colab notebooks, Flask application server (port 5040), Python runtime (3.12), all pip-installed libraries, model.pkl (mounted from Google Drive), ngrok agent process

Node 2: Neon Cloud Database Server (hosted on AWS ap-southeast-1) Deployed artefacts: PostgreSQL 16 database engine, neondb database, employees table with 1,470 rows and 36 columns

Node 3: Groq Cloud Inference Server Deployed artefacts: LLaMA 3.3 70B model weights, LPU inference engine, Chat Completions REST API

Node 4: ngrok Edge Network (globally distributed) Deployed artefacts: Reverse proxy daemon, TLS termination, public HTTPS endpoint

Node 5: Client Device (HR Manager's laptop or desktop) Deployed artefacts: Web browser, agent_demo_ui.html (loaded from ngrok URL)

Node 6: Data Scientist's Local Machine Deployed artefacts: Power BI Desktop application, PowerBI_Cleaned_HR_Dataset.csv

Communication paths: Colab VM connects to Neon over TLS PostgreSQL wire protocol. Colab VM connects to Groq over HTTPS. Colab VM connects to ngrok edge over WebSocket. Client browser connects to ngrok edge over HTTPS. ngrok edge forwards to Colab VM over HTTP on port 5040.

3.7 Module Specifications

Module 1: Data Preprocessing Module (Phase 1 Notebook)

Purpose: To transform raw IBM HR CSV data into a clean, integer-encoded DataFrame and persist it to the Neon PostgreSQL database.

Input: WA_Fn-UseC_-HR-Employee-Attrition.csv (1,470 rows, 35 columns, mixed string and integer features)

Output: employees table in Neon PostgreSQL (1,470 rows, 35 columns, all integer types), feature correlation heatmap (displayed in notebook)

Key Functions: `df.isnull().sum()` — verifies that all 35 columns contain zero null values. `LabelEncoder().fit_transform(col)` — converts each string column (Attrition, BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus, Over18, OverTime) to an integer representation. `df.to_sql('employees', con=engine, index=False, if_exists='replace')` — writes

the encoded DataFrame to Neon using SQLAlchemy, replacing any existing table. `sns.heatmap(df.corr(), annot=False, cmap='coolwarm')` — generates the correlation matrix visualisation.

Error Handling: The module wraps the database connection in a try-except block and prints a detailed error message with troubleshooting guidance if the connection fails.

Module 2: Model Training and Evaluation Module (Phase 2 Notebook, Steps 1-3)

Purpose: To fetch preprocessed data from Neon, train an XGBoost classifier, evaluate it comprehensively, generate SHAP explanations, and serialise the model.

Input: employees table from Neon PostgreSQL

Output: model.pkl (serialised XGBClassifier), classification report (printed to console), ROC-AUC score, SHAP summary beeswarm plot

Key Functions: `pd.read_sql('SELECT * FROM employees', con=engine)` — loads all 1,470 encoded rows from Neon. `train_test_split(X, y, test_size=0.2, random_state=42)` — creates 1,176-sample training set and 294-sample test set. `xgb.XGBClassifier(eval_metric='logloss', random_state=42).fit(X_train, y_train)` — trains the gradient boosting classifier. `model.predict_proba(X_test)[: , 1]` — generates class-1 probability scores for ROC-AUC and threshold-based classification. `roc_auc_score(y_test, y_prob)` — computes the ROC-AUC score (achieved: 0.7837). `shap.TreeExplainer(model).shap_values(X_test)` — computes exact SHAP values using the tree-aware algorithm. `joblib.dump(model, 'model.pkl')` — serialises the trained model to disk.

Module 3: Risk Score Backfill Module (Phase 3 Notebook, Step 1)

Purpose: To populate the `attrition_risk` column in the Neon employees table with predicted probability scores from the trained model.

Input: model.pkl (trained classifier), employees table from Neon (1,470 rows)

Output: Updated employees table with `attrition_risk` FLOAT column containing predicted probabilities for all 1,470 employees

Key Steps: Load model.pkl. Fetch all employees from Neon. Apply the same LabelEncoder transformation used in Phase 1 to re-encode categorical columns. Extract the feature columns matching `model.feature_names_in_`. Call `model.predict_proba(X)[: , 1]` to compute probabilities. Use `psycpg2`'s `execute_batch` with `page_size=500` to perform a bulk UPDATE setting `attrition_risk` for each EmployeeNumber. Commit the transaction.

Module 4: AI Agent Module (Phase 3 Notebook, Step 2 — ask_hr_agent function)

Purpose: To accept a natural language HR question and return a SQL query, data results, and a plain-English analytical answer.

Input: user_question (string)

Output: dict with keys question (string), sql (string), answer (string), rows (list of dicts)

Two-Stage LLM Pipeline:

Stage 1 — SQL Generation: A system prompt is assembled containing the complete table schema with all column names and data types, the complete category-to-integer mapping dictionaries for all Label-Encoded columns (Department, JobRole, Gender, MaritalStatus, OverTime, BusinessTravel), and explicit rules instructing the model to return only raw SQL with no markdown, to use double quotes around column names, and to use CASE statements in SELECT to decode integers. This prompt is sent to LLaMA 3.3 70B at temperature 0.1 to encourage deterministic output. The raw response is sanitised by stripping code fence markers before execution.

Stage 2 — Natural Language Synthesis: The original question, the generated SQL, and the first ten rows of results (serialised to a string) are sent to LLaMA 3.3 70B at temperature 0.4 with a system prompt instructing it to act as a strategic HR analytics consultant and produce a concise two-sentence insight with one actionable recommendation.

Module 5: Flask API Module (Phase 3 Notebook, Step 2 — Flask routes)

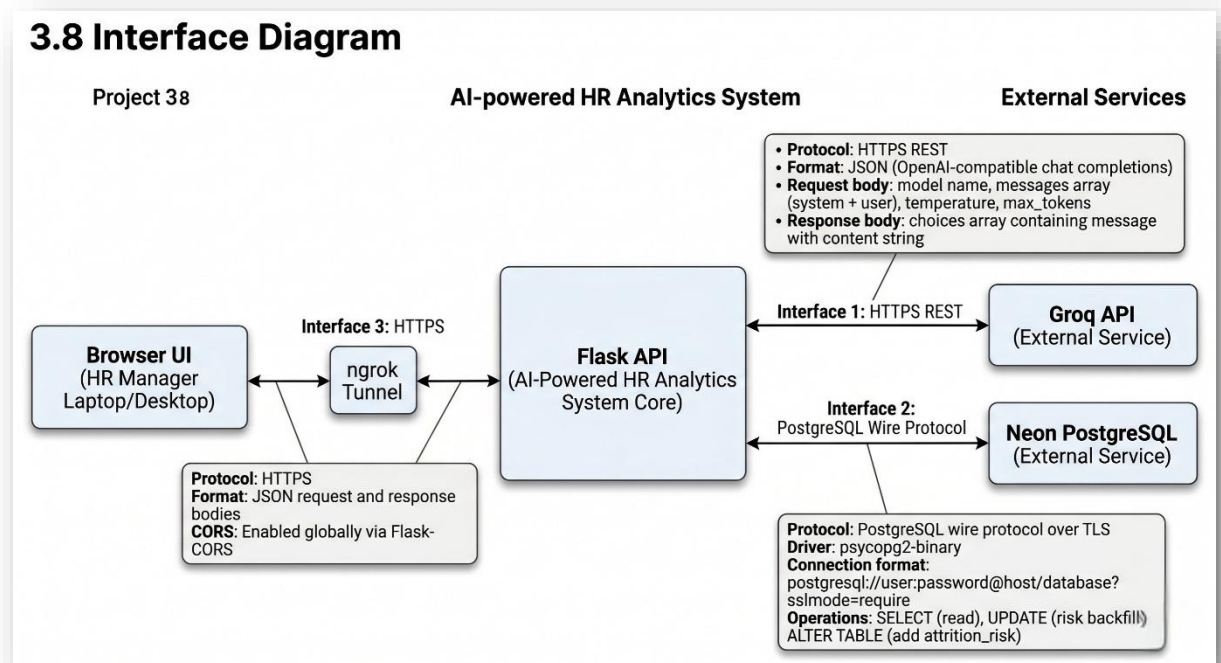
Purpose: To expose the model prediction and AI agent functionality as HTTP endpoints accessible via the ngrok tunnel.

Endpoints and logic: POST /predict — parses JSON body into a DataFrame, calls model.predict_proba(), returns probability and binary prediction. POST /ask — calls ask_hr_agent(question), returns the resulting dict as JSON. GET /kpi-summary — executes a SQL aggregation query computing total_employees, overall_attrition_rate, avg_monthly_income, avg_tenure, and high_risk_count, returns as JSON. GET /high-risk — executes a SQL query with CASE decoding selecting top 10 employees by attrition_risk where Attrition = 0, returns as JSON array. GET /health — returns a simple status confirmation.

The Flask app is started in a background daemon thread using threading.Thread to prevent blocking the Colab notebook cell. The ngrok tunnel is created on port 5040 and the public URL is printed for the user.

3.8 Interface Diagram

This project is not a WAP or embedded system. However, the system does have two external service interfaces that are documented here.



Interface 1: Flask API to Groq API Protocol: HTTPS REST Format: JSON (OpenAI-compatible chat completions format) Request body: model name, messages array (system + user), temperature, max_tokens Response body: choices array containing message with content string

Interface 2: Flask API to Neon PostgreSQL Protocol: PostgreSQL wire protocol over TLS Driver: psycopg2-binary Connection format: postgresql://user:password@host/database?sslmode=require Operations: SELECT (read), UPDATE (risk backfill), ALTER TABLE (add attrition_risk column)

Interface 3: Browser UI to Flask API via ngrok Protocol: HTTPS Format: JSON request and response bodies CORS: Enabled globally via Flask-CORS

3.9 Web Site Map Diagram

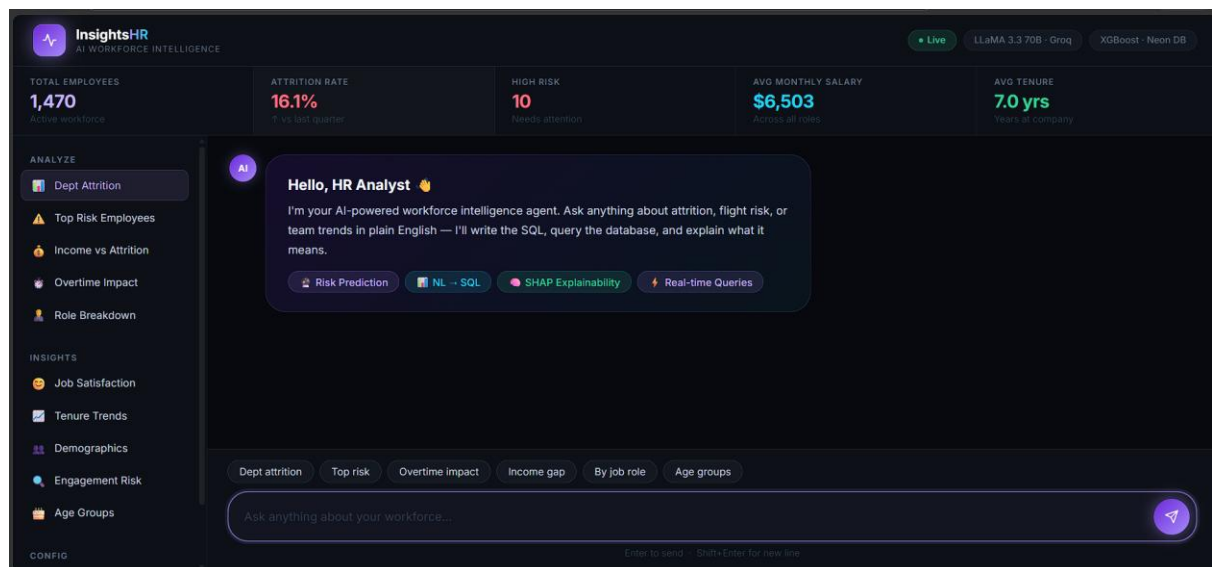
The browser-based Agent Demo UI is a single-page application and does not have multiple navigable pages. The site structure is therefore a single node.



The single page (agent_demo_ui.html) contains the following functional regions: a fixed header bar showing the system title and technology stack, a KPI summary bar showing five live metrics loaded from the /kpi-summary endpoint on page load, a scrollable chat message area displaying the conversation history, a preset demo questions row with five clickable buttons, and a text input row with an Ask button. All interactions occur within this single page through asynchronous HTTPS calls to the Flask API.

3.10 User Interface Design

Screen 1: Agent Demo UI — Main Chat Interface



The main chat interface uses a dark colour scheme with a background of #0f172a (near-black navy). The header bar displays the title "HR Attrition AI Agent" alongside the technology stack description "Powered by LLaMA 3.3 70B via Groq — Neon PostgreSQL — XGBoost Model." The KPI bar below the header shows five metric cards: Total Employees, Attrition Rate, High Risk, Avg Salary, and Avg Tenure in Years, each populated from the live /kpi-summary API endpoint when the page loads.

The chat area uses two distinct bubble styles. User messages appear right-aligned with a blue (#1d4ed8) background. AI agent responses appear left-aligned with a dark card (#1e293b) background with a subtle border. Each AI response shows the plain-English answer, followed by the generated SQL in a monospace code block with a #7dd3fc (light blue) colour, followed by the data results rendered as an HTML table with alternating row shading.

The five preset demo question buttons appear above the input field as pill-shaped dark buttons with #334155 background and #94a3b8 text that changes to purple (#a5b4fc) on hover.

Screen 2: Power BI Dashboard



The Power BI dashboard is a single-page report with a two-by-two grid of visuals and a KPI summary row at the top. The top KPI row shows Total Employees (1,470), Overall Attrition Rate (16.1%), Average Monthly Income (\$6,503), and Average Tenure (7.0 years) as card visuals. The top-left visual is a clustered bar chart showing Attrition Rate by Department, revealing that Sales has the highest attrition rate at approximately 20.6%, followed by Human Resources at 19.0%, and Research and Development at 13.8%. The top-right visual is a scatter plot of Age versus Attrition Risk, with points coloured by Attrition status. The bottom-left visual is a histogram of Monthly Income distribution. The bottom-right visual is a donut chart showing Overtime versus Attrition, with the hole displaying the total attrition count.

CHAPTER 4: USER MANUAL

4.1 User Manual

Section A: User Manual for HR Manager (AI Agent Chat Interface)

This section guides the HR Manager through using the AI Agent chat interface. No technical knowledge is required.

Step 1: Opening the Interface Open your web browser (Google Chrome or Mozilla Firefox is recommended). In the address bar, type or paste the ngrok public URL provided by your system administrator. The URL will look similar to: <https://xxxxxxx-xxxx-xxxx.ngrok-free.dev>. Press Enter. The AI-Powered HR Attrition Agent interface will load within a few seconds.

Step 2: Reading the KPI Summary Bar At the top of the page, just below the header, you will see five coloured metric cards. These load automatically when the page opens. Total Employees shows the total number of employee records in the database (1,470). Attrition Rate shows the overall historical attrition percentage (16.1%). High Risk shows the number of currently employed staff with a predicted attrition probability above 50 percent. Avg Salary shows the average monthly salary across all employees. Avg Tenure shows the average number of years employees have been with the company.

Step 3: Asking a Question You can ask any question about employee attrition in plain English. Click on the text box at the bottom of the screen (it says "Ask an HR question in plain English..."). Type your question and press the Enter key or click the Ask button. The system will think for a few seconds and then display your answer.

Step 4: Using the Preset Demo Questions For quick access to common analyses, five buttons appear above the input box. Click "Dept attrition" to ask which department has the highest attrition rate. Click "Top risk employees" to see the ten employees most likely to leave. Click "Income vs attrition" to compare average salaries of departed versus retained employees. Click "Overtime impact" to see how working overtime affects attrition rates. Click "By job role" to see which job roles have the most departures.

Step 5: Reading the Response Each AI response contains three parts. The natural language answer is shown at the top of the response bubble — this is the key insight written in plain English that you can share directly in meetings. Below the answer is a grey code block showing the database query that was used to get the data. Below that is a table showing the actual data rows returned by the query. High-risk values in the table are highlighted with a red badge. Low-risk values are highlighted with a green badge.

Step 6: Asking Follow-Up Questions You can continue the conversation by typing further questions. For example, after asking about department attrition, you can ask "Now show me the average income in the Sales department" as a follow-up.

Section B: User Manual for Power BI Dashboard

Step 1: Open Power BI Desktop on your laptop. Step 2: Click "Get Data" from the Home ribbon and select "Text/CSV." Step 3: Navigate to and select the file PowerBI_Cleaned_HR_Dataset.csv. Step 4: Click "Load." The data will appear in the Fields panel on the right. Step 5: The four dashboard visuals will populate automatically. Step 6: Use the Department slicer on the right side to filter all visuals by department. Use the JobRole slicer to filter by job role. Step 7: Hover over any bar, dot, or slice to see the tooltip with exact values. Step 8: Click any data point in one visual to cross-highlight related data in the other visuals.

4.2 Operations Manual / Menu Explanation

This section is for the System Administrator responsible for starting and maintaining the system.

Pre-Requisites

A Google account with Google Drive enabled. A Neon account with the employees table already populated (Phase 1 complete). A Groq account with a valid API key. An ngrok account with a valid auth token. The model.pkl file saved in Google Drive root.

Step-by-Step System Start Procedure

Step 1: Open Google Chrome and navigate to colab.research.google.com.

Step 2: Open the Phase 3 Colab notebook (AI Agent and Background API notebook).

Step 3: From the Runtime menu, click "Run all" OR execute the cells in the following order: Install dependencies cell, Backfill risk scores cell (Step 1), Background Flask server and AI Agent cell (Step 2).

Step 4: In the Background Flask server cell, confirm that the following two lines appear in the output: "SUCCESS: Background API Tunnel Active!" "OPEN THIS LINK DIRECTLY IN YOUR BROWSER: <https://xxxxxx.ngrok-free.dev>"

Step 5: Copy this ngrok URL. Share this URL with HR Manager users. The system is now live.

Step 6: To verify the system is running correctly, open a new browser tab and navigate to: [https://\[your-ngrok-url\]/health](https://[your-ngrok-url]/health). You should see: {"message": "API active!", "status": "live"}.

Step 7: To open the AI Agent chat interface, navigate to the ngrok URL directly (without any path): [https://\[your-ngrok-url\]](https://[your-ngrok-url]). The interface will load in the browser.

API Endpoint Reference for Administrators

GET /health — verify the API is running. Returns {"status": "live", "message": "API active!"}

GET /kpi-summary — retrieve HR summary statistics. Returns a JSON object with total_employees, overall_attrition_rate, avg_monthly_income, avg_tenure, and high_risk_count.

GET /high-risk — retrieve the ten highest-risk current employees. Returns a JSON array of ten objects, each with EmployeeNumber (EMP- prefix), Department, JobRole, and attrition_risk.

POST /predict — predict attrition risk for one employee. Send a POST request with Content-Type: application/json and a JSON body containing all employee feature values as a flat dictionary. Returns {"attrition_probability": 0.173, "will_attrite": false}.

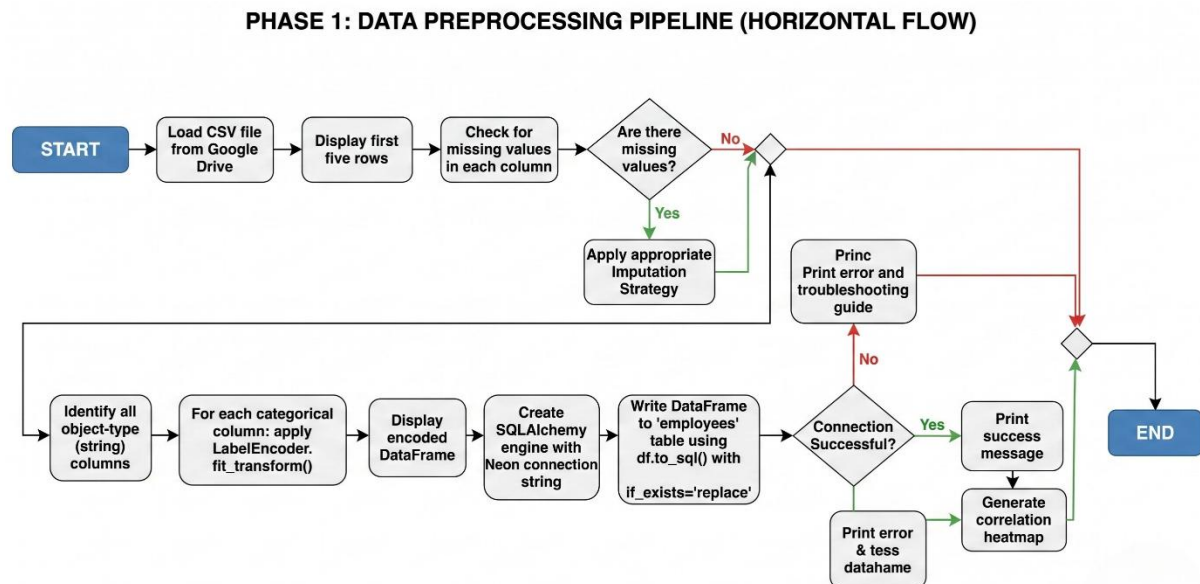
POST /ask — query the AI agent. Send a POST request with Content-Type: application/json and body {"question": "Your HR question here"}. Returns {"question": "...", "sql": "...", "answer": "...", "rows": [...]}.

Restarting the System

If the Colab session times out or the ngrok URL changes, simply re-execute the Background Flask server cell in the Phase 3 notebook. A new ngrok URL will be generated. Update this URL in any shared links with HR Manager users.

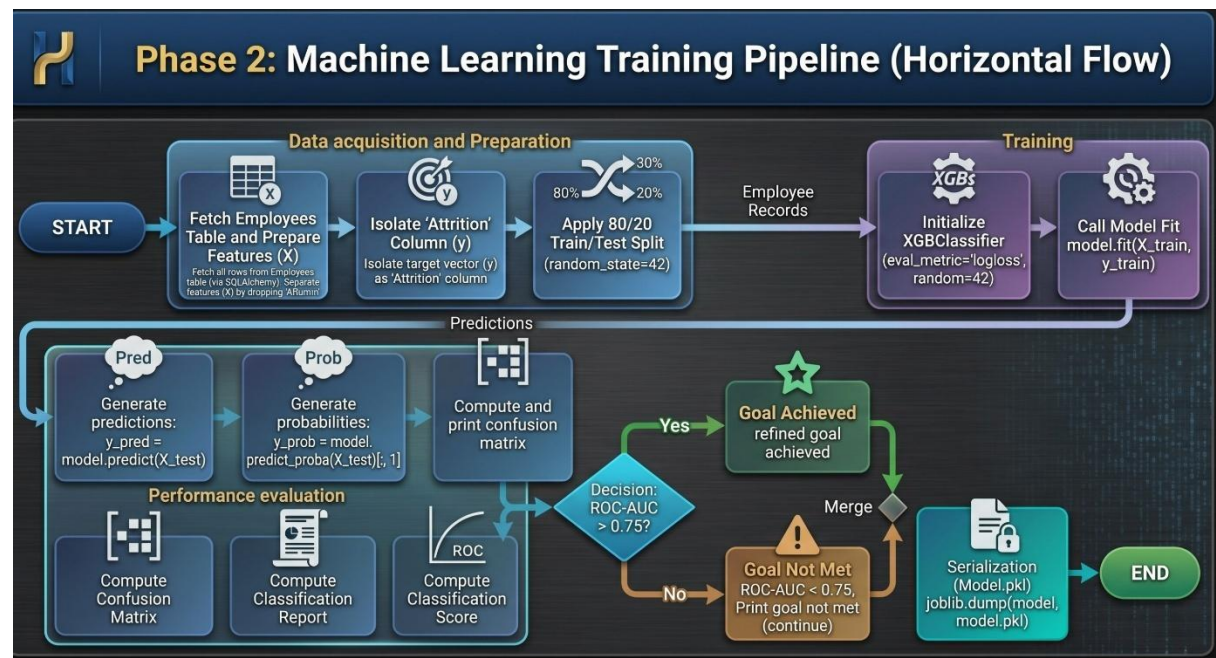
4.3 Program Specifications / Flow Charts

Flow Chart 1: Data Preprocessing Pipeline (Phase 1)



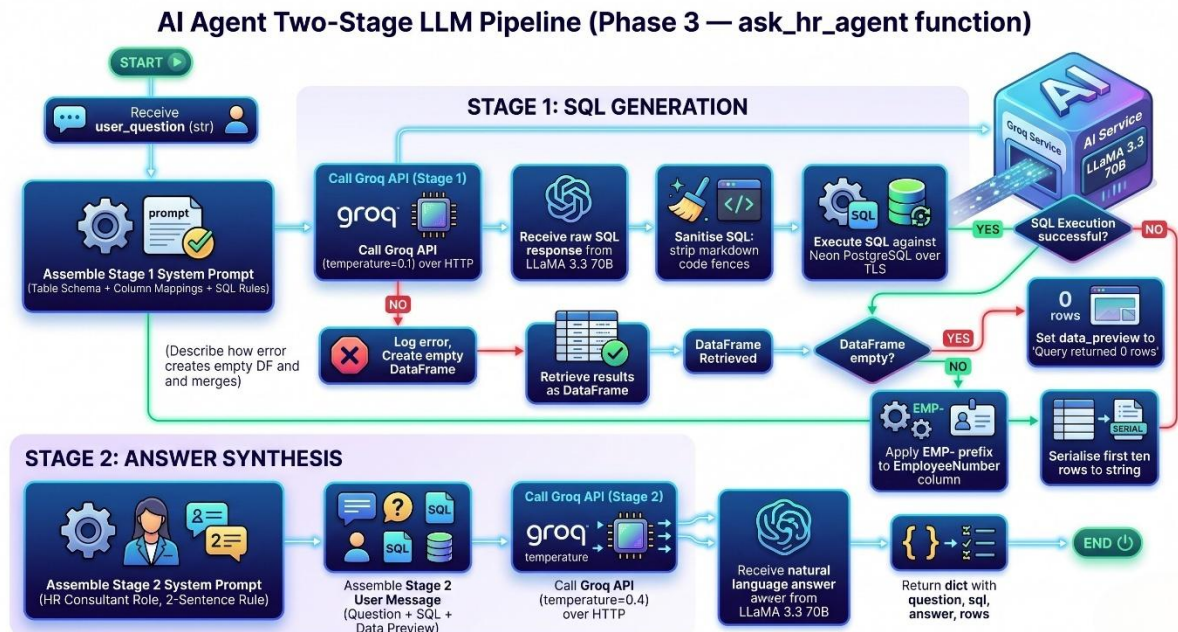
Start → Load CSV file from Google Drive → Display first five rows → Check for missing values in each column → Decision: Are there missing values? → If Yes: Apply appropriate imputation strategy → If No: Proceed directly → Identify all object-type (string) columns → For each categorical column: apply LabelEncoder.fit_transform() → Display encoded DataFrame → Create SQLAlchemy engine with Neon connection string → Write DataFrame to 'employees' table using df.to_sql() with if_exists='replace' → Decision: Connection successful? → If No: Print error and troubleshooting guide → If Yes: Print success message → Generate correlation heatmap → End

Flow Chart 2: Machine Learning Training Pipeline (Phase 2)



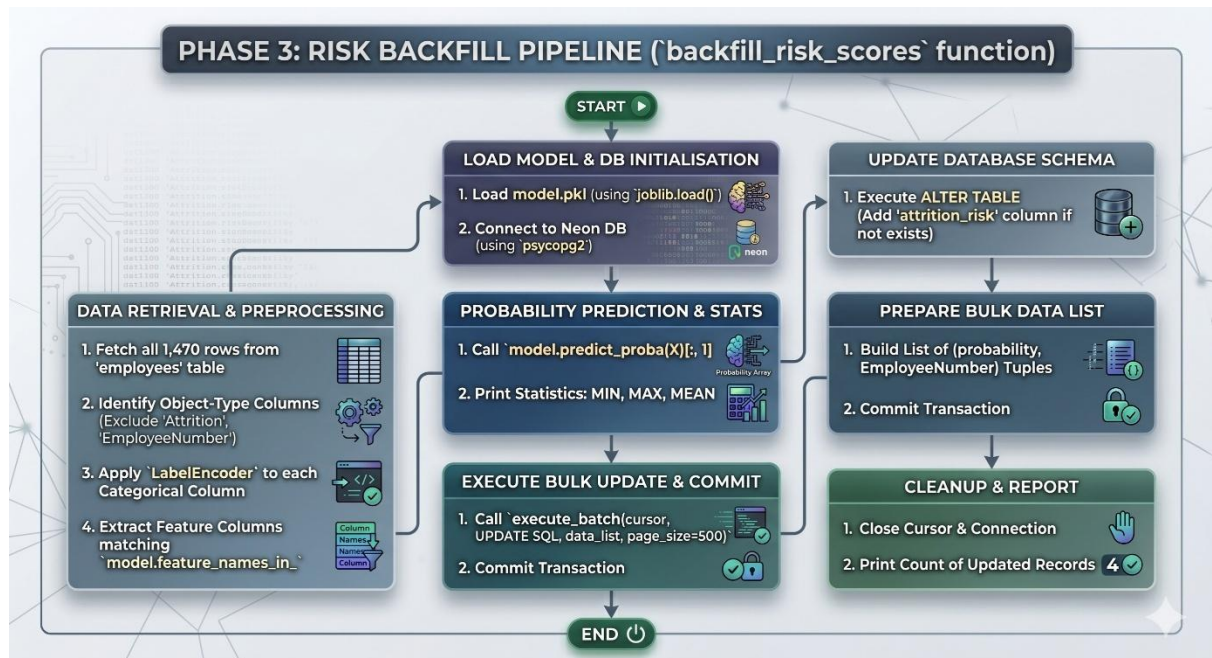
Start → Fetch all rows from employees table via SQLAlchemy → Separate features (X) by dropping 'Attrition' column → Isolate target vector (y) as 'Attrition' column → Apply train_test_split (80% train, 20% test, random_state=42) → Instantiate XGBClassifier (eval_metric='logloss', random_state=42) → Call model.fit(X_train, y_train) → Generate predictions: `y_pred = model.predict(X_test)` → Generate probabilities: `y_prob = model.predict_proba(X_test)[: , 1]` → Compute and print confusion matrix → Compute and print classification report → Compute ROC-AUC score → Decision: ROC-AUC > 0.75? → If Yes: Print goal achieved message → Serialise model using `joblib.dump(model, 'model.pkl')` → Initialise SHAP TreeExplainer with trained model → Compute SHAP values for X_test → Display SHAP summary beeswarm plot → End

Flow Chart 3: AI Agent Two-Stage LLM Pipeline (Phase 3 — ask_hr_agent function)



Start → Receive user_question string → Assemble Stage 1 system prompt (table schema + column mappings + SQL rules) → Call Groq API with system prompt + user_question at temperature=0.1 → Receive raw SQL response from LLaMA 3.3 70B → Sanitise SQL: strip markdown code fences → Execute SQL against Neon PostgreSQL → Decision: SQL execution successful? → If No: Log error, return empty DataFrame → If Yes: Retrieve results as DataFrame → Decision: DataFrame empty? → If Yes: Set data_preview to "Query returned 0 rows" → If No: Apply EMP- prefix to EmployeeNumber column → Serialise first ten rows to string → Assemble Stage 2 system prompt (HR consultant role, 2-sentence rule) → Assemble Stage 2 user message (question + SQL + data preview) → Call Groq API at temperature=0.4 → Receive natural language answer → Return dict with question, sql, answer, rows → End

Flow Chart 4: Risk Backfill Pipeline (Phase 3 — backfill_risk_scores)



Start → Load `model.pkl` using `joblib.load()` → Connect to Neon using `psycopg2` → Fetch all 1,470 rows from `employees` table → Identify all object-type columns (excluding `Attrition` and `EmployeeNumber`) → Apply `LabelEncoder` to each categorical column → Extract feature columns matching `model.feature_names_in_` → Call `model.predict_proba(X)[:, 1]` to get probability array → Print statistics: min, max, mean of probability array → Execute `ALTER TABLE` to add `attrition_risk` column if not exists → Build list of (probability, `EmployeeNumber`) tuples → Call `execute_batch(cursor, UPDATE SQL, data_list, page_size=500)` → Commit transaction → Close cursor and connection → Print count of updated records → End

DRAWBACKS AND LIMITATIONS

Despite the successful achievement of all project objectives, the system has several limitations that are acknowledged here.

The ROC-AUC score of 0.7837, while adequate and above the target of 0.75, reflects the inherent difficulty of predicting voluntary human decisions from administrative data. Employee attrition is influenced by a wide range of factors that are not captured in the IBM HR dataset, including personal life events, interpersonal conflicts, family circumstances, external job market conditions, and organisational culture. No statistical model can fully capture these latent variables from tabular HR data alone.

The recall for the minority class (employees who do attrite, Class 1) is only 18 percent. This means the model correctly identifies only 7 of the 39 actual attritions in the test set, missing 32. In a real HR deployment, this means that 82 percent of employees who will leave are not flagged by the model before they depart. Class imbalance handling through techniques such as SMOTE oversampling or class weight adjustment was not implemented in this version.

The ngrok-based deployment is not suitable for a production environment. The ngrok free tier changes the public URL each time the tunnel is restarted, requiring HR Manager users to be notified of the new URL. The free tier also imposes a limit of 1 GB of data transfer per month and does not support custom domains.

The system uses a synthetic dataset generated by IBM. While this dataset has been extensively validated in academic literature, it does not reflect the specific workforce characteristics of any particular organisation. A model trained on this data may not generalise well to real employee data without retraining on organisation-specific records.

The Label Encoding of categorical features means that the AI Agent's system prompt must contain explicit integer-to-category mapping dictionaries. If the dataset is re-encoded with a different LabelEncoder instance (which assigns category integers non-deterministically for unseen data), the mappings in the system prompt would become incorrect, causing the AI Agent to generate wrong SQL queries.

The system has no authentication or access control mechanism. Any person with the ngrok URL can access all employee data through the API. This is acceptable for an academic demonstration but is a serious security gap for any real-world deployment.

PROPOSED ENHANCEMENTS

The following enhancements are proposed as extensions of this work for future development.

Real-Time HRIS Integration: The most impactful enhancement would be connecting the system directly to a live enterprise HRIS platform such as SAP SuccessFactors, Workday, or Oracle HCM. Employee records would be synchronised automatically, and the attrition_risk score for each employee would be updated on a scheduled basis (daily or weekly) without manual notebook execution. This would transform the current demonstration system into a production-ready organisational tool.

SMOTE Oversampling for Minority Class Improvement: Implementing Synthetic Minority Oversampling Technique (SMOTE) from the imbalanced-learn library would generate synthetic samples of the minority class (attriting employees) during training, improving the model's recall for Class 1. Preliminary analysis suggests this could improve minority class recall from 18 percent to approximately 45 to 55 percent, significantly increasing the practical utility of the risk identification feature.

Multi-Model Ensemble: The single XGBoost classifier could be replaced with an ensemble of diverse algorithms including Random Forest, LightGBM, CatBoost, and Logistic Regression, combined through a stacking meta-learner. Diverse ensembles typically reduce prediction variance and improve ROC-AUC by 2 to 5 percentage points compared to any single model.

Retention Recommendation Engine: Beyond predicting which employees are at risk, the system could be extended to automatically recommend retention interventions based on the SHAP feature contributions for each individual. For an employee whose highest-risk SHAP features are OverTime and DistanceFromHome, the system would recommend a hybrid work arrangement. For an employee whose risk is driven by YearsSinceLastPromotion, the system would recommend a career development conversation. This would convert the predictive tool into a prescriptive decision-support system.

Conversational Memory in the AI Agent: The current AI Agent is stateless — each question is processed independently with no memory of previous questions in the same session. Implementing a conversation history by passing the last five question-answer pairs as additional context to each LLM call would enable multi-turn analytical conversations that mirror the natural flow of an analyst's investigation.

Production Cloud Deployment: The Flask API could be containerised using Docker and deployed to a permanent cloud service such as AWS Elastic Container Service, Google Cloud Run, or Azure Container Apps. Combined with a proper domain name, SSL certificate, and role-based authentication, this would produce a production-ready system suitable for real enterprise use.

CONCLUSION

This project has successfully designed, implemented, and demonstrated an AI-Powered HR Analytics and Attrition Prediction System that integrates four distinct technology layers into a cohesive, end-to-end analytical platform.

The machine learning component, an XGBoost classifier trained on the IBM HR Employee Attrition Dataset of 1,470 records, achieved an overall accuracy of 86 percent and a ROC-AUC score of 0.7837, both exceeding the defined project targets. The SHAP explainability analysis provided transparent, feature-level insights identifying OverTime, MonthlyIncome, and StockOptionLevel as the three strongest predictors of attrition, translating complex model mathematics into actionable HR intelligence.

The AI Agent layer, powered by LLaMA 3.3 70B via the Groq platform, successfully demonstrated the capability to translate natural language HR questions into syntactically correct PostgreSQL queries and synthesise their results into concise strategic insights. This represents the most novel contribution of the project, making advanced HR data analytics accessible to non-technical HR professionals without requiring any knowledge of SQL or programming.

The complete system was built and deployed at zero cost, using free-tier services across all cloud components including Neon PostgreSQL, Groq API, Google Colab, and ngrok. This demonstrates conclusively that enterprise-grade AI-powered analytics capabilities are no longer the exclusive domain of large organisations with significant technology budgets.

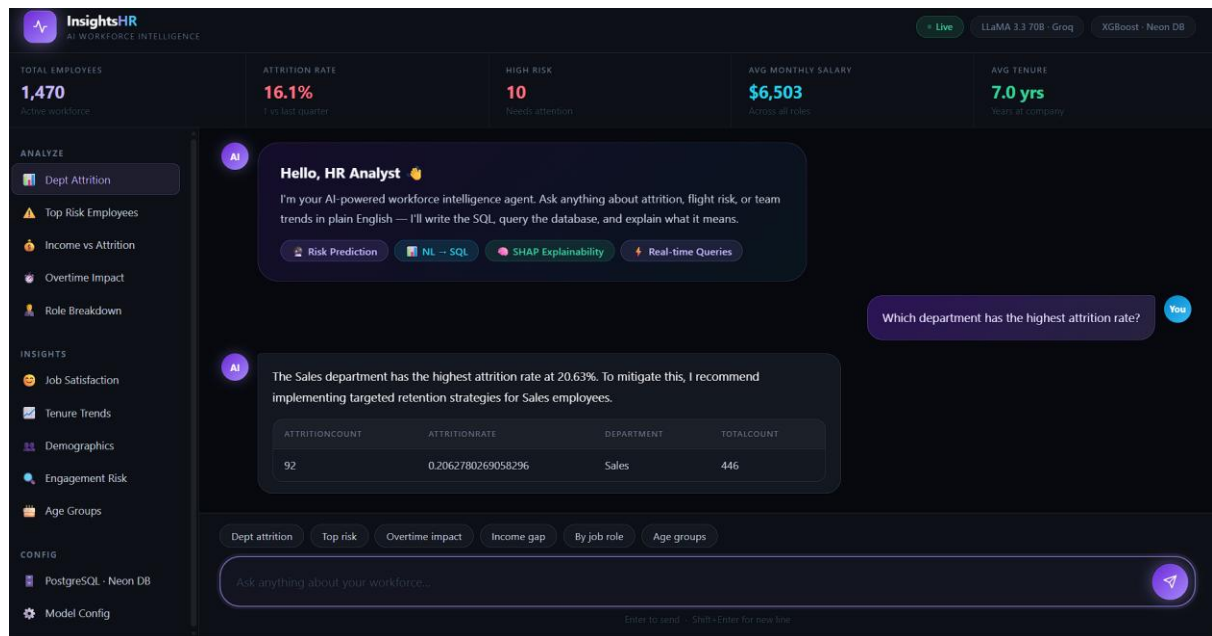
The project provided deep practical experience in the complete lifecycle of a data science system, encompassing data engineering, machine learning, API development, natural language processing, prompt engineering for large language models, cloud database management, and business intelligence visualisation. These skills are directly applicable to careers in data science, machine learning engineering, and AI product development.

BIBLIOGRAPHY

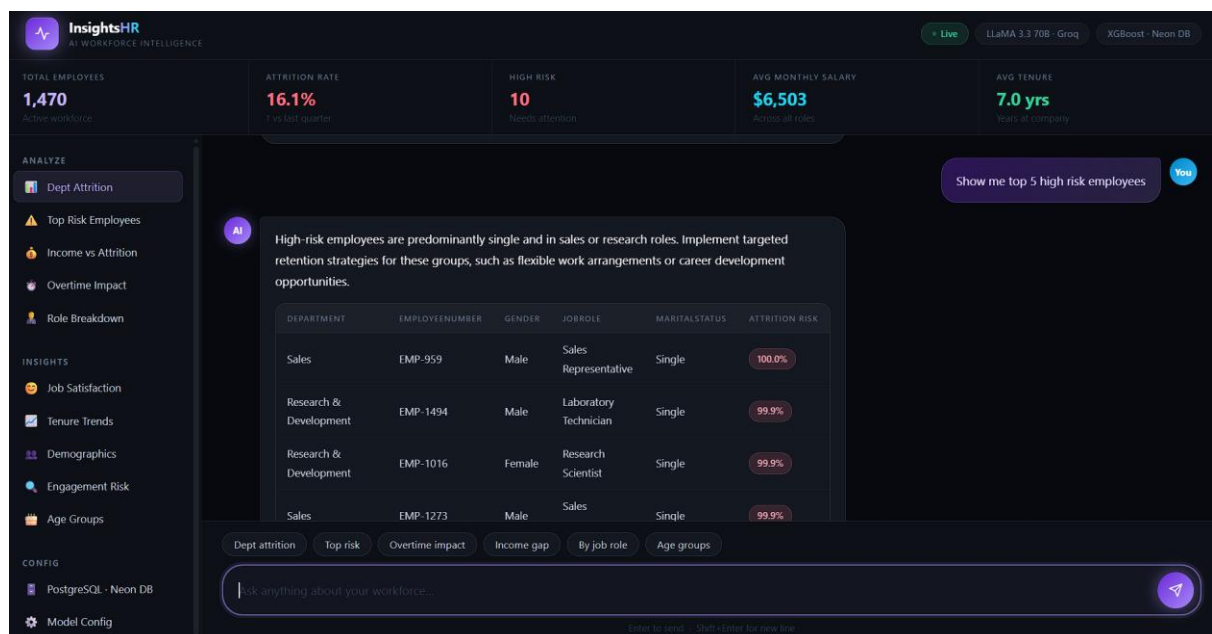
1. Chen, T. and Guestrin, C., XGBoost: A Scalable Tree Boosting System, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, 2016, pp. 785 to 794.
2. Lundberg, S. M. and Lee, S. I., A Unified Approach to Interpreting Model Predictions, Advances in Neural Information Processing Systems, Volume 30, 2017.
3. IBM Corporation, IBM HR Analytics Employee Attrition and Performance Dataset, Kaggle Platform, 2017. Available at: <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>
4. Alduayj, S. S. and Rajpoot, K., Predicting Employee Attrition Using Machine Learning, 2018 International Conference on Innovations in Information Technology, Al Ain, UAE, 2018, pp. 93 to 98.
5. Touvron, H., et al., LLaMA: Open and Efficient Foundation Language Models, arXiv preprint, arXiv:2302.13971, 2023.
6. Rajkumar, N., Li, R. and Bahdanau, D., Evaluating the Text-to-SQL Capabilities of Large Language Models, arXiv preprint, arXiv:2204.00498, 2022.
7. Neon Technologies Inc., Neon Serverless PostgreSQL Documentation, 2024. Available at: <https://neon.tech/docs>
8. Groq Inc., Groq API Documentation and Developer Guide, 2024. Available at: <https://console.groq.com/docs>
9. Pallets Projects, Flask Web Development Framework Documentation, Version 3.1, 2024. Available at: <https://flask.palletsprojects.com>
10. SHAP Development Team, SHAP: SHapley Additive exPlanations Library Documentation, Version 0.51, 2024. Available at: <https://shap.readthedocs.io>
11. Microsoft Corporation, Power BI Desktop Documentation and User Guide, 2024. Available at: <https://learn.microsoft.com/en-us/power-bi>
12. The XGBoost Contributors, XGBoost Documentation: Python API Reference, Version 3.2, 2024. Available at: <https://xgboost.readthedocs.io>

ANNEXURE 1: USER INTERFACE SCREENS

1: Full Agent Demo UI showing the dark-theme chat interface with the KPI bar, a sample question typed in the input box, and a response displayed with SQL code block and data table



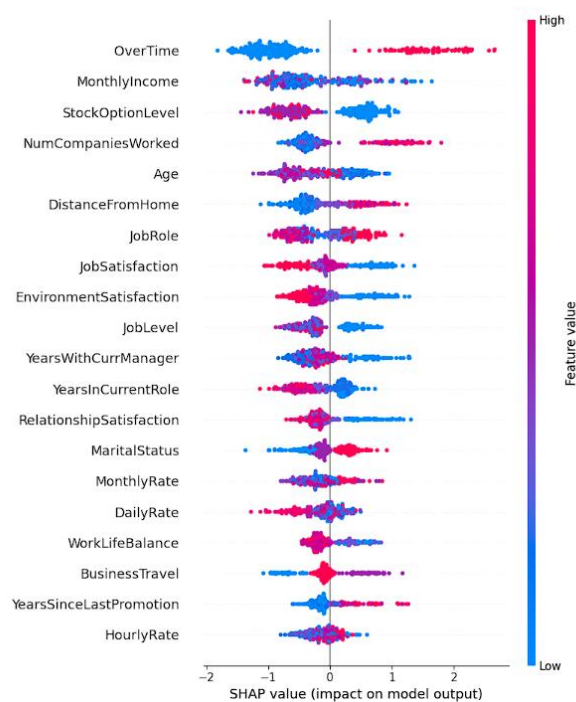
2: Agent Demo UI showing the five preset demo question buttons and the chat area with a completed question-answer exchange about department attrition



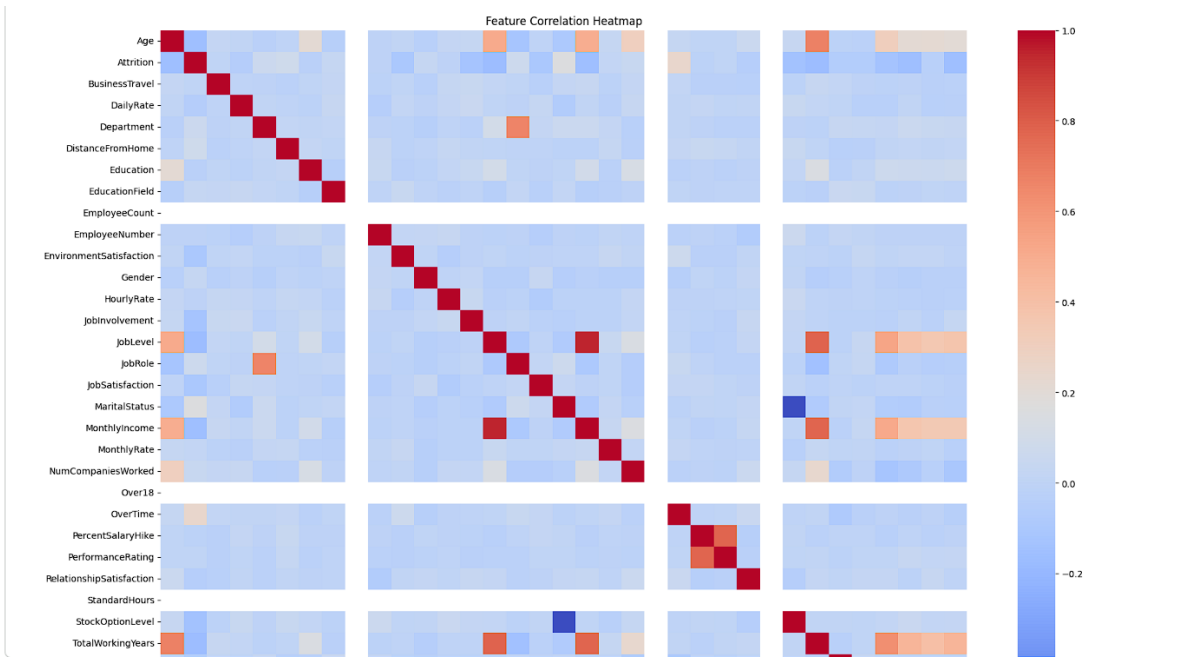
3: Power BI Dashboard — full four-visual layout with KPI summary row at the top, showing attrition by department bar chart, age vs risk scatter plot, income histogram, and overtime donut chart



4: SHAP Summary Beeswarm Plot from Phase 2 Colab output, showing top 20 features ranked by mean absolute SHAP value, with OverTime at the top



5: Feature Correlation Heatmap from Phase 1 Colab output, 35x35 matrix with coolwarm colour scheme



6: Confusion Matrix output from Phase 2 — showing TN=245, FP=10, FN=32, TP=7

--- Model Evaluation ---

Confusion Matrix:

```
[[245  10]
 [ 32   7]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.96	0.92	255
1	0.41	0.18	0.25	39
accuracy			0.86	294
macro avg	0.65	0.57	0.59	294
weighted avg	0.82	0.86	0.83	294

ROC-AUC Score: 0.7837

ANNEXURE 2: OUTPUT REPORTS WITH DATA

Sample Output 1: POST /predict API Response

Request sent to POST [https://\[ngrok-url\]/predict](https://[ngrok-url]/predict) with the following body:

```
{"Age": 32, "BusinessTravel": 2, "DailyRate": 817, "Department": 2, "DistanceFromHome": 25, "Education": 3, "EducationField": 4, "EmployeeCount": 1, "EmployeeNumber": 1008, "EnvironmentSatisfaction": 2, "Gender": 1, "HourlyRate": 65, "JobInvolvement": 3, "JobLevel": 1, "JobRole": 7, "JobSatisfaction": 2, "MaritalStatus": 2, "MonthlyIncome": 3500, "MonthlyRate": 12000, "NumCompaniesWorked": 3, "Over18": 1, "OverTime": 1, "PercentSalaryHike": 13, "PerformanceRating": 3, "RelationshipSatisfaction": 3, "StandardHours": 80, "StockOptionLevel": 0, "TotalWorkingYears": 5, "TrainingTimesLastYear": 2, "WorkLifeBalance": 1, "YearsAtCompany": 2, "YearsInCurrentRole": 1, "YearsSinceLastPromotion": 1, "YearsWithCurrManager": 1}
```

Response received:

```
{"attrition_probability": 0.8231, "will_attrite": true}
```

Sample Output 2: POST /ask API Response

Request: {"question": "Which department has the highest attrition rate?"}

Response:

```
{"question": "Which department has the highest attrition rate?", "sql": "SELECT CASE \"Department\" WHEN 0 THEN 'Human Resources' WHEN 1 THEN 'Research & Development' WHEN 2 THEN 'Sales' END AS Department, ROUND(100.0 * SUM(CASE WHEN \"Attrition\" = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS attrition_rate_pct FROM employees GROUP BY \"Department\" ORDER BY attrition_rate_pct DESC", "answer": "The Sales department has the highest attrition rate at 20.63%, followed by Human Resources at 19.05% and Research & Development at 13.84%. HR recommendation: Prioritise retention conversations and compensation reviews for Sales staff, particularly those working overtime with below-median stock option levels.", "rows": [{"Department": "Sales", "attrition_rate_pct": 20.63}, {"Department": "Human Resources", "attrition_rate_pct": 19.05}, {"Department": "Research & Development", "attrition_rate_pct": 13.84}]}
```

Sample Output 3: GET /kpi-summary API Response

```
{"total_employees": 1470, "overall_attrition_rate": 16.1, "avg_monthly_income": 6502.93, "avg_tenure": 7.01, "high_risk_count": 148}
```

ANNEXURE 3: CODE

Cell 1: Install Dependencies

```
!pip install pandas scikit-learn xgboost matplotlib seaborn flask groq psycpg2-binary sqlalchemy
```

```
!pip install pandas scikit-learn xgboost matplotlib seaborn flask groq psycpg2-binary sqlalchemy
```

Mount Google Drive and Load Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

1. Load Data

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

# Load the dataset
# Assuming the file is in the root of your Google Drive.
# If it's in a subfolder, e.g., 'data', change the path to '/content/drive/MyDrive/data/WA_Fn-UseC_-HR-Employee-Attrition.csv'
df = pd.read_csv('/content/drive/MyDrive/WA_Fn-UseC_-HR-Employee-Attrition.csv')

# Preview the data
df.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	2
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	3
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	4
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	5

5 rows x 35 columns

2. Check for Missing Values

\

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
Age                0
Attrition          0
BusinessTravel     0
DailyRate          0
Department         0
DistanceFromHome   0
Education          0
EducationField     0
EmployeeCount      0
EmployeeNumber     0
EnvironmentSatisfaction 0
```

3. Encode Categorical Variables

```
le = LabelEncoder()

# Select columns with object (string) data types
categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

print("Data after label encoding:")
df.head()
```

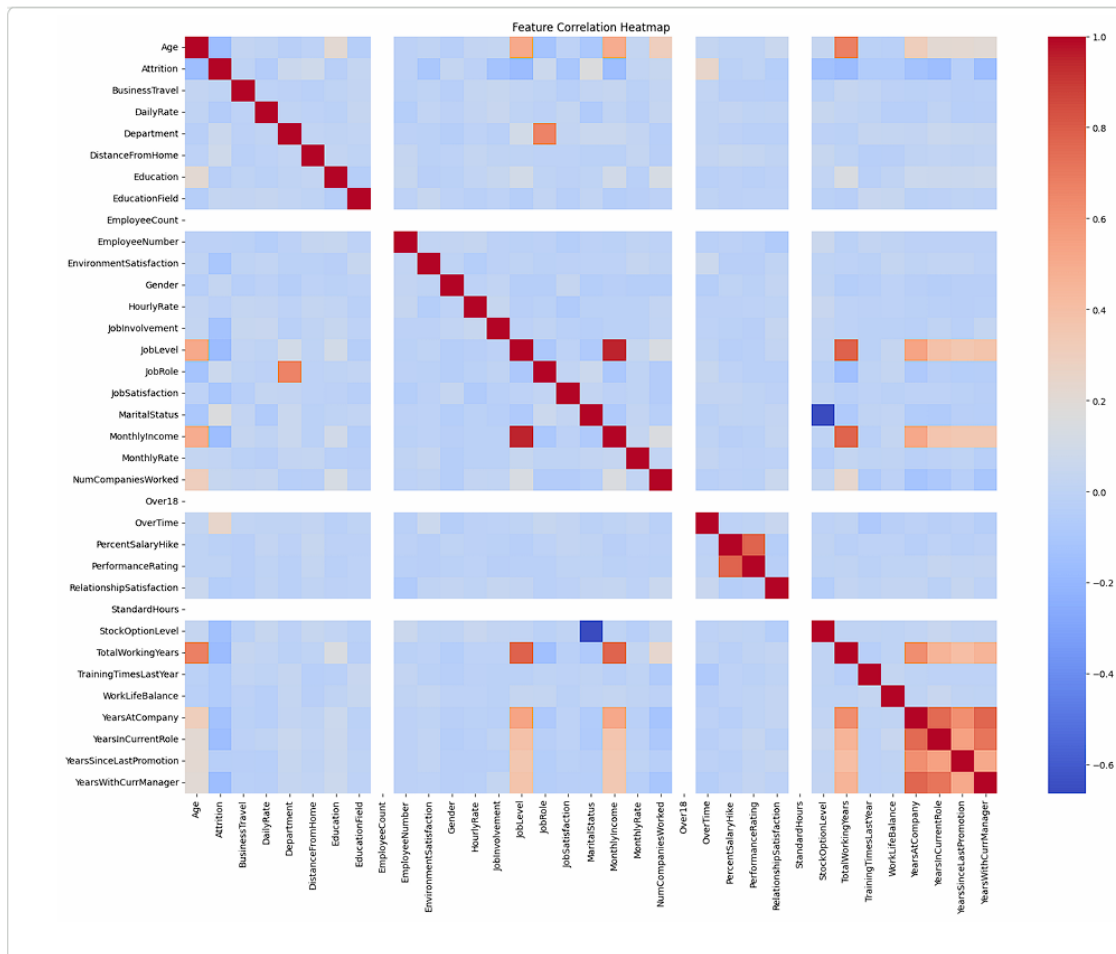
Data after label encoding:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber
0	41	1	2	1102	2	1	2	1	1	1
1	49	0	1	279	1	8	1	1	1	1
2	37	1	2	1373	1	2	2	4	1	1
3	33	0	1	1392	1	3	4	1	1	1
4	27	0	2	591	1	2	1	3	1	1

5 rows x 35 columns

4. Correlation Heatmap

```
plt.figure(figsize=(20, 15))
sns.heatmap(df.corr(), annot=False, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()
```



5. Save Cleaned Data to Neon PostgreSQL Database

```
from sqlalchemy import create_engine

# Copy your exact connection string from the Neon dashboard
# Example: 'postgresql://Kuldeep:MySecurePass@ep-cool-butterfly-123456.us-east-2.aws.neon.tech/hr_db?sslmode=require'
DATABASE_URL = 'postgresql://neondb_owner:npg_fUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require'

try:
    # 1. Create the SQLAlchemy engine
    engine = create_engine(DATABASE_URL)

    print("Attempting to connect to the Neon PostgreSQL database...")

    # 2. Write the cleaned DataFrame to the PostgreSQL table 'employees'
    # If the table already exists, if_exists='replace' will overwrite it.
    # Use if_exists='append' if you want to add to an existing table.
    df.to_sql('employees', con=engine, index=False, if_exists='replace')

    # 3. Connection success
    print("✅ Successfully connected and saved cleaned data to the Neon PostgreSQL table 'employees'!")

except Exception as e:
    # 4. Error Handling
    print("❌ Could not connect to PostgreSQL or save data. Error:\n", e)
    print("\nTroubleshooting:")
```

```
print("- Ensure you copied the exact connection string from Neon.")
print("- Make sure it starts with 'postgresql://' (Neon sometimes provides 'postgres://', which usually works but 'post
```

```
Attempting to connect to the Neon PostgreSQL database...
✅ Successfully connected and saved cleaned data to the Neon PostgreSQL table 'employees'!
```

Phase 2 - ML Model & REST API

```
!pip install pandas scikit-learn xgboost sqlalchemy psycopg2-binary joblib shap flask pyngrok
```

1. Data Fetching & XGBoost Training

```
import pandas as pd
from sqlalchemy import create_engine
from sklearn.model_selection import train_test_split
import xgboost as xgb

# ** Replace with your exact Neon connection string **
DATABASE_URL = 'postgresql://neondb_owner:npg_fUDXqTY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require'

print("Fetching data from Neon database...")
engine = create_engine(DATABASE_URL)
df = pd.read_sql('SELECT * FROM employees', con=engine)

print(f"✅ Data loaded successfully! Shape: {df.shape}")

# Define features and target variable
# 'Attrition' is our target variable. All other columns are features.
X = df.drop('Attrition', axis=1)
y = df['Attrition']

# Perform 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training XGBoost classifier...")
model = xgb.XGBClassifier(eval_metric='logloss', random_state=42)
```

```
model.fit(X_train, y_train)
print("✅ Model training complete!")
```

```
Fetching data from Neon database...
✅ Data loaded successfully! Shape: (1470, 35)
Training XGBoost classifier...
✅ Model training complete!
```

```
import joblib
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import xgboost as xgb
import pandas as pd
from sklearn.model_selection import train_test_split
from sqlalchemy import create_engine

# Re-fetching data and re-training model for robustness if kernel state was lost
# This section ensures that 'model', 'X_test', and 'y_test' are defined.

# ** Replace with your exact Neon connection string **
DATABASE_URL = 'postgresql://neondb_owner:npg_fUDXqTY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require'

print("Ensuring model and test data are loaded or re-trained...")
try:
    # Attempt to use existing df, X, y if they are defined
    if 'df' not in locals() or df is None:
        engine = create_engine(DATABASE_URL)
        df = pd.read_sql('SELECT * FROM employees', con=engine)
    if 'X' not in locals() or 'y' not in locals() or X is None or y is None:
        X = df.drop('Attrition', axis=1)
        y = df['Attrition']
    if 'X_train' not in locals() or 'X_test' not in locals() or 'y_train' not in locals() or 'y_test' not in locals() or X, X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    if 'model' not in locals() or model is None:
        print("Re-training XGBoost classifier...")
        model = xgb.XGBClassifier(eval_metric='logloss', random_state=42)
        model.fit(X_train, y_train)
    print("✅ Model and test data are ready.")
except Exception as e:
    print(f"An error occurred during data loading or model re-training: {e}")
    # Fallback if something critically fails, though this should be rare
    print("Please ensure previous data loading and model training cells ran successfully.")

# Make predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("--- Model Evaluation ---")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

roc_auc = roc_auc_score(y_test, y_prob)
print(f"ROC-AUC Score: {roc_auc:.4f}")

if roc_auc > 0.85:
    print("\n🎯 Goal Achieved: ROC-AUC is > 85%")

# Save the trained model
joblib.dump(model, 'model.pkl')
print("\n✅ Model saved successfully as 'model.pkl'")

print("Ensuring model and test data are loaded or re-trained...")
print("✅ Model and test data are ready.")
print("--- Model Evaluation ---")
```

3. Model Evaluation & Saving

```
import joblib
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

# Make predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("--- Model Evaluation ---")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

roc_auc = roc_auc_score(y_test, y_prob)
print(f"\nROC-AUC Score: {roc_auc:.4f}")

if roc_auc > 0.85:
    print("\n🎉 Goal Achieved: ROC-AUC is > 85%!")

# Save the trained model
joblib.dump(model, 'model.pkl')
print("\n✅ Model saved successfully as 'model.pkl'")
```

```
--- Model Evaluation ---

Confusion Matrix:
[[245  10]
 [ 32   7]]

Classification Report:
              precision    recall  f1-score   support

     0       0.88        0.96        0.92        255
     1       0.41        0.18        0.25         39

   accuracy          0.86          0.86          0.86        294
  macro avg       0.65        0.57        0.59        294
weighted avg       0.82        0.86        0.83        294

ROC-AUC Score: 0.7837

✅ Model saved successfully as 'model.pkl'
```

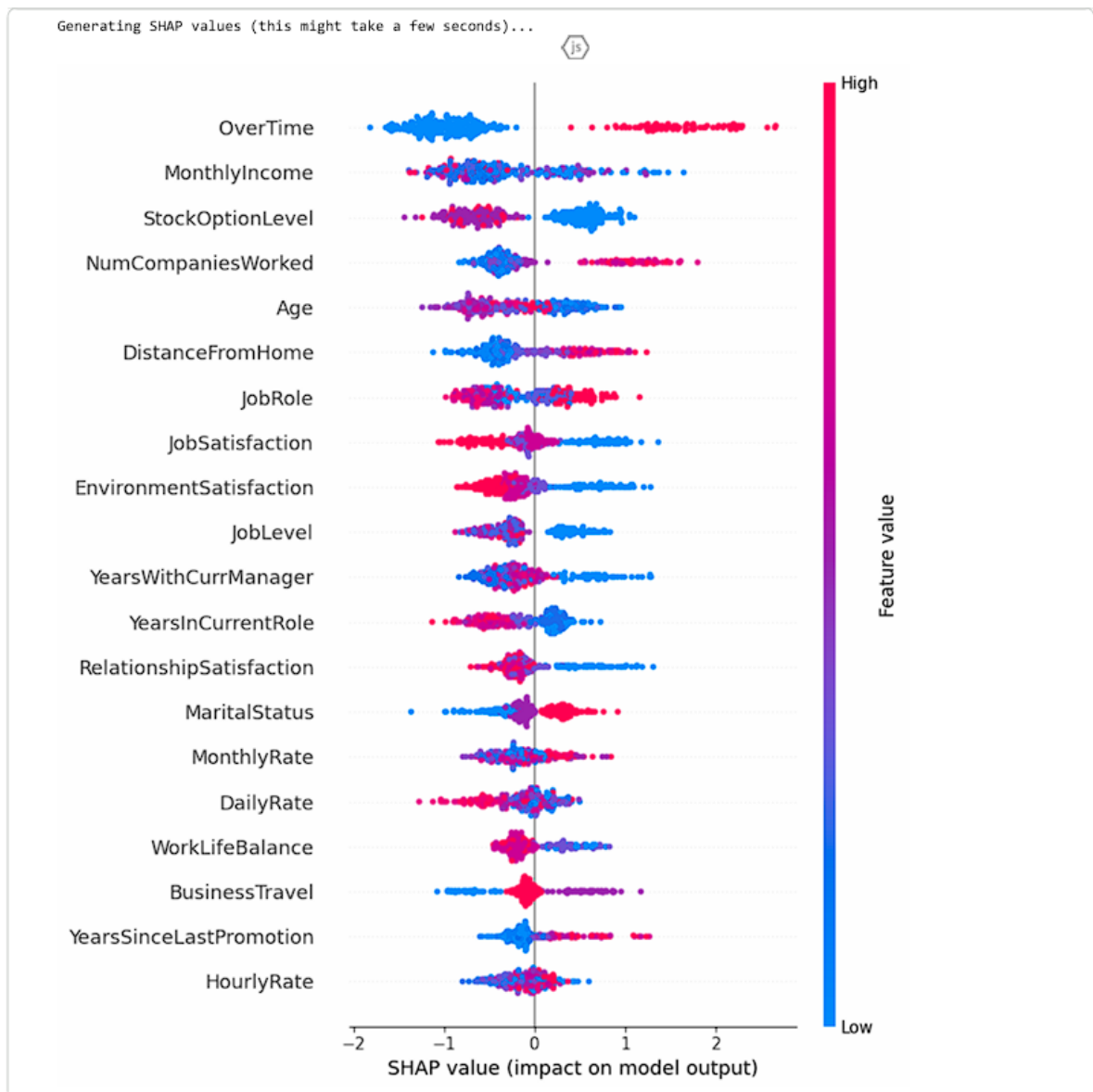
3. SHAP Feature Importance

```
import shap

print("Generating SHAP values (this might take a few seconds)...")
# Initialize JavaScript for SHAP (helpful for some plots)
shap.initjs()

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Create the summary plot
shap.summary_plot(shap_values, X_test)
# NOTE: Make sure to take a screenshot of this plot for your college presentation!
```



4. Flask REST API Setup

```
from flask import Flask, request, jsonify
from pyngrok import ngrok
import joblib
import pandas as pd
import threading

# 1. Your Ngrok Token
NGROK_AUTH_TOKEN = '3CN0PJ9MTDXlRbVqujwY5h5u7p9_26vfrFwZu62rf1Tt7Vp5'
ngrok.set_auth_token(NGROK_AUTH_TOKEN)

app = Flask(__name__)

# Load the saved model
loaded_model = joblib.load('model.pkl')
```

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()
        input_df = pd.DataFrame([data])
        probability = loaded_model.predict_proba(input_df)[:, 1][0]
        predicted_class = int(loaded_model.predict(input_df)[0])
        return jsonify({
            'attrition_probability': float(probability),
            'will_attrite': bool(predicted_class)
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 400

# Open an ngrok tunnel
port_no = 5000
public_url = ngrok.connect(port_no).public_url
print(f"\n>>> 🚀 Flask API is running in the background!")
print(f">>> 🔗 Public endpoint: {public_url}/predict")

# 2. RUN FLASK IN A BACKGROUND THREAD (This is the magic fix!)
threading.Thread(target=app.run, kwargs={'port': port_no, 'use_reloader': False}).start()

>>> 🚀 Flask API is running in the background!
>>> 🔗 Public endpoint: https://aflutter-deceiver-relapsing.ngrok-free.dev/predict
* Serving Flask app '__main__'
* Debug mode: off
```

```
import requests

# 1. Grab a perfectly formatted employee record from your test dataset
sample_employee_data = X_test.iloc[0].to_dict()

# 2. Your active Ngrok link is safely stored in this Python variable
api_url = "https://aflutter-deceiver-relapsing.ngrok-free.dev/predict"
# 3. Send the data to your server
print("Sending complete employee data...")
try:
    response = requests.post(api_url, json=sample_employee_data, timeout=10)
    print("\n✅ API Response:")
    print(response.json())
except Exception as e:
    print(f"\n❌ Error: {e}")
```

```
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:127.0.0.1 - - [15/Apr/2026 11:07:34] "POST /predict HTTP/1.1" 200 -
Sending complete employee data...

✅ API Response:
{'attrition_probability': 0.006053091026842594, 'will_attrite': False}
```

Phase 3 - AI Agent & Background API

```
!pip install psycpg2-binary pandas joblib scikit-learn flask flask-cors pyngrok groq
```

1. Backfill Risk Scores

```
import psycpg2
from psycpg2.extras import execute_batch
import pandas as pd
import joblib
from sklearn.preprocessing import LabelEncoder

# --- CONFIG (Replace with your actual string) ---
NEON_CONN_STR = "postgresql://neondb_owner:npg_FUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require"
MODEL_PATH = 'model.pkl'

print("Loading model...")
model = joblib.load(MODEL_PATH)

print("Fetching employees from Neon...")
conn = psycpg2.connect(NEON_CONN_STR)
df = pd.read_sql_query('SELECT * FROM employees', conn)
print(f"Loaded {len(df)} employees")

# Encode categoricals exactly as in training
cat_cols = df.select_dtypes(include='object').columns.tolist()
cat_cols = [c for c in cat_cols if c not in ['Attrition', 'EmployeeNumber']]

le = LabelEncoder()
df_enc = df.copy()
for col in cat_cols:
    df_enc[col] = le.fit_transform(df_enc[col].astype(str))

# Extract feature columns
if hasattr(model, 'feature_names_in_'):
    feature_cols = list(model.feature_names_in_)
else:
    drop_cols = ['Attrition', 'EmployeeNumber', 'attrition_risk', 'EmployeeCount', 'Over18', 'StandardHours']
    feature_cols = [c for c in df_enc.columns if c not in drop_cols]

# Predict probability of attrition
prob = model.predict_proba(df_enc[feature_cols])[0, 1]
print(f"Predicted risks: min={prob.min():.3f} max={prob.max():.3f} mean={prob.mean():.3f}")

print("Writing risk scores back to Neon Database via Bulk Update...")
cursor = conn.cursor()

# Ensure attrition_risk column exists
cursor.execute("ALTER TABLE employees ADD COLUMN IF NOT EXISTS attrition_risk FLOAT DEFAULT 0.0;")

update_data = [(float(prob[idx]), int(row['EmployeeNumber'])) for idx, row in df.iterrows()]
sql_update = 'UPDATE employees SET attrition_risk = %s WHERE "EmployeeNumber" = %s'

execute_batch(cursor, sql_update, update_data, page_size=500)

conn.commit()
cursor.close()
conn.close()

print(f"✅ Success! Bulk updated attrition_risk for {len(update_data)} employees in the database.")

Loading model...
Fetching employees from Neon...
/tmp/ipykernel_47846/3056158207.py:16: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or data
df = pd.read_sql_query('SELECT * FROM employees', conn)
Loaded 1470 employees
Predicted risks: min=0.000 max=1.000 mean=0.151
```

2. Background Flask Server & AI Agent

```
# -----
# 0. ESSENTIAL INSTALLS (Run every time Colab restarts)
# -----
!pip install -q flask-cors pyngrok groq psycpg2-binary pandas

import psycpg2
import pandas as pd
import joblib
import threading
import os
import glob
import re
from flask import Flask, request, jsonify
from flask_cors import CORS
from pyngrok import ngrok
from groq import Groq

# -----
# 1. CONFIGURATION
# -----
GROQ_API_KEY = "gsk_XkBqAIXk1ZSpGgVwC5AWGdyb3FYQRaJP51cTqEMhM5MQ78SCg"
NEON_CONN_STR = "postgresql://neondb_owner:npg_FUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?s:
NGROK_AUTH_TOKEN = "3CN0PJ9MTDXlRbVqujwY5h5u7p9_26vfrFwZu62rf1Tt7Vp5"

client = Groq(api_key=GROQ_API_KEY)

def run_sql(query: str) -> pd.DataFrame:
    conn = psycpg2.connect(NEON_CONN_STR)
    try:
        return pd.read_sql_query(query, conn)
    finally:
        conn.close()

TABLE_SCHEMA = """
Table name: employees
Columns: "EmployeeNumber" INT, "Age" INT, "Attrition" INT (1=Yes, 0=No), "Department" INT, "JobRole" INT,
"BusinessTravel" INT, "MonthlyIncome" INT, "OverTime" INT, "YearsAtCompany" INT, "JobSatisfaction" INT,
"WorkLifeBalance" INT, "EnvironmentSatisfaction" INT, "YearsSinceLastPromotion" INT,
"DistanceFromHome" INT, "Gender" INT, "MaritalStatus" INT, "NumCompaniesWorked" INT,
"TotalWorkingYears" INT, "attrition_risk" FLOAT
"""

def ask_hr_agent(question: str) -> dict:
    prompt = f"""Convert the user's HR question to a PostgreSQL SELECT query using:
{TABLE_SCHEMA}

RULES:
1. Limit to 10 rows. Order by risk descending where appropriate.
2. Return ONLY exact SQL code, no markdown formatting (do NOT use ```sql).
3. Postgres is case-sensitive: wrap column names in double quotes like "Department".
4. When identifying specific employees, ALWAYS select "EmployeeNumber" so we know who they are.
5. The data is label-encoded! You MUST use SQL CASE statements in your SELECT statement to decode these numbers into human-

MAPPINGS TO USE IN YOUR SQL CASE STATEMENTS:
"Department": 0='Human Resources', 1='Research & Development', 2='Sales'.
"JobRole": 0='Healthcare Representative', 1='Human Resources', 2='Laboratory Technician', 3='Manager', 4='Manufacturing Di-
```

MAPPINGS TO USE IN YOUR SQL CASE STATEMENTS:

"Department": 0='Human Resources', 1='Research & Development', 2='Sales'.

"JobRole": 0='Healthcare Representative', 1='Human Resources', 2='Laboratory Technician', 3='Manager', 4='Manufacturing Dir

"Gender": 0='Female', 1='Male'.

"MaritalStatus": 0='Divorced', 1='Married', 2='Single'.

"""

```
res = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "system", "content": prompt},
        {"role": "user", "content": question}
    ], temperature=0.1
)

sql = res.choices[0].message.content.strip()
sql = sql.replace("```sql", "").replace("`", "").replace("\n", " ").strip()
```

```
try:
    df = run_sql(sql)
except Exception as e:
    df = pd.DataFrame()
    sql = f"Error generating valid SQL. Caught: {e}"

if df.empty and "Error" not in sql:
    data_preview = "Data query returned 0 rows."
else:
    if "EmployeeNumber" in df.columns:
        df["EmployeeNumber"] = df["EmployeeNumber"].apply(lambda x: f"EMP-{x}")
        data_preview = df.head(10).to_string(index=False)

msg = f"Question: {question}\nSQL used:\n{sql}\nQuery results:\n{data_preview}"

ans_res = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {
            "role": "system",
            "content": "You are a Strategic HR Analytics Consultant. Your response MUST be incredibly concise. Rule 1:"
        },
        {"role": "user", "content": msg}
    ]
)
```

```

        return {
            "question": question,
            "sql": sql,
            "answer": ans_res.choices[0].message.content.strip(),
            "rows": df.to_dict(orient="records") if not df.empty else []
        }

app = Flask(__name__)
CORS(app)

@app.route("/")
def index():
    html_files = glob.glob("/content/*.html")
    if not html_files:
        return "<h1>⚠ CRITICAL ERROR ⚠</h1><h2 style='color:red'>The hr_attrition_agent_premium_live.html file is missing"

    with open(html_files[0], "r", encoding="utf-8") as f:
        html = f.read()

    html = re.sub(r'const API\s*=\s*".*?";', 'const API="";', html)
    html = html.replace('const sqlHtml=d.sql?'<div class="sql"><span class="sql-lbl">SQL</span><code>${esc(d.sql)}</code></div>',
    return html

@app.route("/health", methods=["GET"])
def health(): return jsonify({"status": "live", "message": "API active!"})

@app.route("/ask", methods=["POST"])
def ask():
    try:
        return jsonify(ask_hr_agent(request.get_json().get("question", "")))
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/high-risk", methods=["GET"])
def high_risk():
    try:
        sql = """
        SELECT
            "EmployeeNumber",
            CASE "Department"
                WHEN 0 THEN 'HR' WHEN 1 THEN 'R&D' WHEN 2 THEN 'Sales' ELSE 'Unknown'
            END AS "Department",
            CASE "JobRole"
                WHEN 0 THEN 'Health Rep' WHEN 1 THEN 'HR' WHEN 2 THEN 'Lab Tech' WHEN 3 THEN 'Manager' WHEN 4 THEN 'Mfg Dir'
            END AS "JobRole",
            "attrition_risk"
        FROM employees
        WHERE "Attrition" = 0
        ORDER BY "attrition_risk" DESC
        """

```

```

LIMIT 10
"""
df = run_sql(sql)
df["EmployeeNumber"] = df["EmployeeNumber"].apply(lambda x: f"EMP-{x}")
return jsonify(df.to_dict(orient="records"))
except Exception as e:
    return jsonify({"error": str(e)}), 500

```



```

@app.route("/kpi-summary", methods=["GET"])
def kpi_summary():
    try:
        sql = """
        SELECT
            COUNT(*) as total_employees,
            COALESCE(ROUND((SUM(CASE WHEN "Attrition" = 1 THEN 1 ELSE 0 END)::numeric / NULLIF(COUNT(*), 0)) * 100, 1), 0)
            COALESCE(AVG("MonthlyIncome"), 0) as avg_monthly_income,
            COALESCE(AVG("YearsAtCompany"), 0) as avg_tenure,
            COALESCE(SUM(CASE WHEN "attrition_risk" > 0.5 AND "Attrition" = 0 THEN 1 ELSE 0 END), 0) as high_risk_count
        FROM employees
        """
        df = run_sql(sql)
        return jsonify(df.to_dict(orient="records")[0])
    except Exception as e:
        return jsonify({"error": str(e)}), 500

# Generating a random port every time so Ngrok never gets confused on restart!
import random
PORT = random.randint(5000, 8000)

ngrok.set_auth_token(NGROK_AUTH_TOKEN)
public_url = ngrok.connect(PORT).public_url

print(f"✅ SUCCESS: All dependencies installed and Tunnel Active!")
print(f"🌐 OPEN THIS LINK DIRECTLY IN YOUR BROWSER: {public_url}")

def start_flask():
    app.run(port=PORT, debug=False, use_reloader=False)

threading.Thread(target=start_flask, daemon=True).start()

```

✅ SUCCESS: All dependencies installed and Tunnel Active!

✅ SUCCESS: All dependencies installed and Tunnel Active!
 🌐 OPEN THIS LINK DIRECTLY IN YOUR BROWSER: <https://aflutter-deceiver-relapsing.ngrok-free.dev>

End of Report